

南通大學



本科毕业设计

题 目	基于 LLM 的智能家居控制器设计与实现
--------	----------------------

学生姓名： 胡阳

专 业： 人工智能

指导教师： 梁惺彦

完成日期： 2026 年 05 月 10 日

诚信承诺书

本人承诺：所呈交的毕业设计是本人在导师指导下进行的研究成果。除了文中特别加以标注和致谢的地方外，不包含其他人已发表或撰写过的研究成果。撰写论文过程中不存在使用人工智能（AI）工具直接生成论文核心内容、虚构或篡改数据、规避论文查重检测等行为。参与同一工作的其他同志对本研究所做的任何贡献均已在文中作了明确的说明并表示了谢意。

签 名： 古阳 日 期： 2026年5月10日

本论文使用授权说明

本人完全了解南通大学有关保留、使用学位论文的规定，即：学校有权保留论文及送交论文复印件，允许论文被查阅和借阅；学校可以公布论文的全部或部分内容。

（保密的论文在解密后应遵守此规定）

学生签名： 古阳 指导教师签名： 梁峰彦 日期： 2026年5月10日

摘 要

随着物联网技术的发展，智能家居逐渐融入到人们的日常生活中，然而传统的智能家居系统大多仍然依靠预设规则来控制设备，在自然语言理解、模糊表达解析、多轮交互等各方面还存在一定不足，难以满足用户越来越口语化、多样化的需求。近些年来，大语言模型在语义理解方面具有较强的能力，也给智能家居控制方式的改善带来了新的思路。

本文以智能家居语义控制场景为设计对象。在分析国内外有关研究现状的基础上，对大语言模型语义解析、设备通信、语音交互等技术进行了整理。设计出“五层三端”协同架构，把云端、用户端、设备端分为用户交互层、LLM 推理层、数据存储层、设备管理与驱动层、硬件执行层五个层次进行管理。

在系统设计中，LLM 推理层通过调用云端大语言模型接口完成语义解析，并结合 Prompt 工程实现结构化控制指令生成。系统采用 JSON 格式进行数据传输与设备通信，同时加入关键词过滤与语义校验双层安全检验，以提高系统运行的安全性与稳定性。

在系统实现上，用 ESP32 和 STM32 构建了原型，可以实现语音输入、语义解析、设备控制以及基础自然语言交互等功能。测试结果说明，在测试集上系统的指令解析准确率为 92%，平均响应时间为 1.8s。在模糊语义以及多轮对话的环境之下，系统会维持较为稳定的解析成果。

本文完成了基于大语言模型的智能家居语义控制系统的实现，验证了智能家居场景下大语言模型的可行性。

关键词：大语言模型，智能家居，ESP32，STM32

ABSTRACT

With the development of Internet of Things technology, smart homes have gradually entered people's daily life. However, traditional smart home systems still mainly rely on preset rules to control devices, and they still have some shortcomings in natural language understanding, fuzzy expression recognition, and multi-turn interaction. It is difficult for them to meet users' increasingly conversational and diverse interaction needs. In recent years, large language models have shown strong abilities in semantic understanding, which also provides a new idea for improving smart home control methods.

This paper takes the smart home semantic control scenario as the research object. Based on the analysis of related research at home and abroad, technologies related to large language model semantic parsing, device communication, and voice interaction are studied and summarized. A "five-layer and three-terminal" collaborative architecture is designed. The cloud side, user side, and device side are divided into five layers, including the user interaction layer, LLM reasoning layer, data storage layer, device management and driver layer, and hardware execution layer.

In the system design, the LLM reasoning layer completes semantic parsing by calling cloud large language model APIs, and generates structured control commands with the help of Prompt engineering. The system uses JSON format for data transmission and device communication. At the same time, keyword filtering and semantic validation are added as a double security checking mechanism to improve the security and stability of the system.

In the system implementation, a prototype based on ESP32 and STM32 is built, which can realize voice input, semantic parsing, device control, and basic natural language interaction. Test results show that the command parsing accuracy of the system reaches 92% on the test dataset, and the average response time is 1.8 seconds. Under fuzzy semantic expressions and multi-turn dialogue conditions, the system can still maintain relatively stable parsing performance.

This paper completes the implementation of a smart home semantic control system based on large language models, and verifies the feasibility of large language models in smart home scenarios.

Keywords: Large Language Models (LLMs); Smart Home; ESP32; STM32

目 录

摘 要..... I

ABSTRACT..... II

目 录..... III

第一章 引言..... 1

1.1 研究背景与意义 1

1.2 国内外研究现状 1

1.3 本文主要研究工作与创新点 2

1.4 论文组织结构 3

第二章 相关技术基础..... 5

2.1 大语言模型（LLM）定义与技术架构 5

2.1.1 大语言模型的定义与核心架构基础..... 5

2.1.2 大语言模型的完整训练流程 5

2.1.3 大语言模型面向智能家居场景的核心技术特性..... 6

2.2 智能家居关键技术 6

2.2.1 设备通信与管理技术..... 7

2.2.2 语音交互技术（ASR/TTS） 8

第三章 系统需求分析..... 9

3.1 功能需求分析 9

3.1.1 自然语言模糊指令解析功能 9

3.1.2 上下文感知多轮对话功能 9

3.1.3 语音交互功能 10

3.2 可扩展性需求分析 10

3.3 可靠性需求分析 11

3.4 需求可行性分析 11

第四章 系统设计..... 13

4.1 系统架构设计 13

4.1.1 分层架构总体设计..... 13

4.1.2 用户交互层设计.....	13
4.1.3 云端大语言模型推理层设计	14
4.1.4 设备管理与驱动层设计	15
4.1.5 硬件执行层设计.....	15
4.1.6 数据存储层设计.....	15
4.1.7 系统数据流与控制流程设计	16
4.2 核心模块详细设计	17
4.2.1 基于大语言模型 API 的意图解析模块设计.....	17
4.2.2 安全机制与异常指令拦截模块设计.....	18
4.2.3 设备控制执行模块设计	19
4.2.4 MQTT 通信机制模块设计.....	20
第五章 系统实现.....	22
5.1 系统硬件实现.....	22
5.1.1 系统整体硬件架构设计	22
5.1.2 硬件连接与实物实现.....	22
5.1.3 ESP32 模块	23
5.1.4 STM32 模块	23
5.1.5 传感器与执行设备.....	24
5.1.6 ESP32 与 STM32 通信方式	24
5.2 软件设计实现.....	25
5.2.1 意图解析功能实现.....	25
5.2.2 安全校验与异常指令拦截实现	27
5.2.3 语音交互功能实现.....	28
5.2.4 MQTT 功能实现	30
第六章 系统测试与分析.....	32
6.1 测试环境与测试方案.....	32
6.1.1 测试环境搭建	32
6.1.2 测试方案设计	33
6.2 功能测试结果与分析.....	34
6.3 性能测试结果与分析.....	37

6.4 本章小结.....	38
第七章 总结与展望.....	39
7.1 研究工作总结	39
7.2 研究不足与未来展望.....	40
参考文献.....	42
致谢.....	44

第一章 引言

1.1 研究背景与意义

随着物联网技术的发展，智能家居系统逐渐融入人们的日常生活，成为智慧生活的重要组成部分。

近些年来，随着移动互联网、云计算、人工智能、无线通信技术的发展，智能家居产业规模不断扩大，相关产品也从单一设备智能化向全屋互联、场景协同方向发展。智能音箱、智能照明、环境监测、安防监控、智能家电等产品被广泛使用，使得家庭环境中设备的联网数量越来越多，用户对智能化生活方式的接受程度也不断提高。

通过将家居设备接入网络，实现设备之间的信息互联与协同控制，用户可以完成远程操控和调节，提升居住环境舒适性，同时推动传统家居向智能化、网络化方向发展^[1]。

但是目前大部分智能家居系统依然采用基于规则的方式进行控制。通常依靠预设的指令模板来完成识别，在使用过程中需要按照固定的表达式进行操作^[2]。一旦表达方式发生变化，系统就难以理解用户的意图，会影响到控制效果。多轮交互场景中，此类方法缺少对上下文信息的利用能力，对话连续性较差^[3]。

近些年来，大语言模型（Large Language Model, LLM）在自然语言处理方面取得明显进步，语义理解、上下文建模等方面都有较好的能力^[4]。与传统方法相比，LLM 对于复杂的语句有着更加深入的语义解析能力，能根据上下文信息推测出用户的意图^[5]。因此，把大语言模型应用于智能家居系统中，系统可以实现语义理解，提高对自然语言输入的适应性^[6]。

在此背景下，对智能家居场景下的语义驱动控制方法进行研究，具有一定的研究价值与现实意义。一方面可以改善用户和系统之间的交互体验，同时也为智能家居系统的进一步发展提供了新的实现思路。

1.2 国内外研究现状

国外对于智能家居语义控制技术的研究起步较早，研究体系相对完善。早期方法主要依赖规则匹配与传统机器学习模型实现语义解析，例如基于支持向量机（Support Vector Machine, SVM）、条件随机场（Conditional Random Field, CRF）等方法进行意图识别与槽位填充。随着深度学习的发展，预训练的语言模型被广泛应用到语义理解的任务中，在标

准数据集上通过微调的方式可以获得较高的分类准确率^[7]。该类方法在结构化指令、单轮交互场景中表现相对较为稳定，但是因为它依赖于事先定义好的意图类别，难以适应开放语义环境。当用户输入有模糊表达、隐含语义或者跨设备控制需求时，系统难以准确解析。另外，多轮对话场景中缺少上下文建模，无法保证语义的一致性，进而影响整个交互体验^[8]。

近些年来，随着大语言模型的发展，国外研究逐渐转向基于生成式模型智能家居控制方法^[9]。该类方法可以直接从自然语言生成控制指令，语义理解能力优于传统方法。但是它在实际应用中还存在着计算开销大、对云端依赖性强、系统实时性差等缺陷，多设备协同控制、系统工程化实现方面仍有待进一步优化。

国内对于智能家居语义交互技术的发展历程大致可分为关键词匹配、深度学习模型以及大语言模型驱动三个阶段。早期主要是模板匹配的方式来进行语音控制，交互的灵活性较低；后来用 BERT 等模型的语义解析方法在一定程度上提高了理解能力，但是仍然受到固定标签体系的限制^[9]。近几年来，一些研究采用提示工程或者模型微调的方式处理模糊语义，取得了一定的成果。但是目前的研究大多集中于算法层面，缺少完整的系统设计以及真实的测试，而且对于多设备协同控制、边缘部署等工程问题的考虑较少。

就整体而言，目前研究在语义理解的效果同系统实际应用之间仍难以实现良好平衡。怎样在保证语义解析能力的同时提高系统的运行效率，实现工程化落地，成了亟待解决的问题之一。

1.3 本文主要研究工作与创新点

针对传统智能家居控制系统语义理解能力差、复杂指令解析能力不足、系统工程化部署困难等问题，本文主要研究大语言模型在智能家居控制系统中的应用方法，结合嵌入式平台完成系统实现，主要研究工作如下。

（1）面向智能家居的语义约束指令解析方法设计

针对传统规则或者关键词匹配的方式难以应对自然语言中模糊表达、多轮对话、多种指令等状况，本文采用一种面向智能家居场景的语义约束指令解析方法。该方法采用 Prompt 约束方式引导大语言模型在语义解析过程中按照结构化的输出规则，再用关键词辅助校验对解析结果进行二次约束，从而实现自然语言到结构化控制指令的稳定映射，在提高语义理解能力的同时提高系统输出结果的可控性、可靠性。

（2）云边协同的智能家居分层控制架构设计

针对大语言模型计算资源消耗大、难以直接部署到嵌入式设备上这一问题，设计出一种云端语义解析和边缘设备执行相结合的分层协同架构。云端负责语义理解、意图解析工作，边缘侧负责设备的通信、控制指令执行，实现计算任务和控制任务的分开部署，既保证语义理解的能力，又提高系统的响应速度和实时性，满足智能家居场景下低延迟交互的要求。

（3）基于双层机制的智能家居安全控制策略实现

针对大语言模型生成结果的不确定性以及潜在异常指令风险，使用一种基于关键词规则拦截与语义安全判定的双层安全控制机制。通过智能家居高危指令关键词规则库对异常指令进行快速过滤，并结合安全约束 Prompt 机制从语义层面对潜在风险指令进行识别，从多角度进行拦截异常指令，提高系统的整体安全性、鲁棒性。

（4）系统实现与实验验证

以 ESP32、STM32 硬件平台为基础，完成智能家居控制系统软硬件一体化的实现，构建起完整的语音交互、设备控制原型系统。用多种类型的自然语言指令测试集对系统的语义解析能力、响应性能进行评价。实验结果表明，本文提出的方法对复杂的语义解析任务有较高的准确率，且实时响应较好，说明本文提出的方法具有一定的工程可行性。

（5）开放式自然语言交互功能实现

除设备控制功能外，系统还实现了基础开放式自然语言交互能力。在非控制类输入场景下，大语言模型可根据用户输入生成对应文本回复，并结合 TTS 模块完成语音反馈，实现基础智能助手式交互功能，提高系统人机交互体验。

1.4 论文组织结构

本文共分为七个章节，各章节安排如下。

第一章介绍研究背景与意义，分析智能家居语义控制的发展现状，并总结国内外相关研究进展及存在问题。

第二章介绍系统实现过程中涉及的关键技术，包括大语言模型相关原理、语义解析方法以及嵌入式控制与通信技术等内容。

第三章对系统需求进行分析，明确系统核心功能需求并进行可行性分析。

第四章对系统总体架构进行设计，分析各模块功能及系统运行流程。

第五章详细阐述系统各核心模块的实现方法。

第六章对系统功能与性能进行测试与分析。

第七章总结全文并对未来研究方向进行展望。

第二章 相关技术基础

2.1 大语言模型（LLM）定义与技术架构

2.1.1 大语言模型的定义与核心架构基础

大语言模型（Large Language Model, LLM）是以 Transformer 架构为基础的自然语言处理模型，其主要特点是利用自注意力机制分析文本中各个词语之间的依赖关系，从而理解上下文语义^[10]。相比于传统的人工规则、特征工程、浅层机器学习的方法，大语言模型经过大量的语料训练之后，可以自动从语料中提取出语言特征、语义信息，对于复杂的语境理解、文本生成等表现出较强能力^[11]。

LLM 是根据已有的上下文内容推测最有可能出现的下一个 Token。当模型处理大量的文本数据时，它会逐步形成对语法、语义等特征的认识。随着训练数据量以及轮次的增加，模型对于自然语言的理解能力也逐渐增强，可应用于文本生成，语义理解等领域。

2.1.2 大语言模型的完整训练流程

大语言模型的完整训练流程是由数据输入层、大语言模型训练层、模型核心能力层、落地应用层四个部分构成的，如图 2.1 所示。

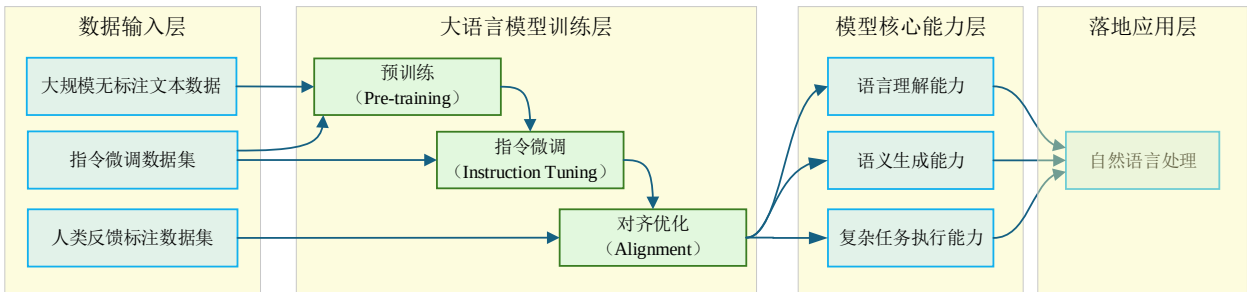


图 2.1 大语言模型的完整训练流程示意图

其中，大语言模型的训练层通常包括预训练、指令微调、对齐优化这三个阶段^[12]：

（1）预训练（Pre-training）：通过大规模语料对模型进行训练，使其具备一定基础的语言理解与表示能力^[11]。训练数据一般来源于各种大规模无标注文本数据，包括互联网内容、书籍资料等各方面的语料。模型经过大量文本样本的学习之后，逐渐掌握一些常见的语义规律，为之后任务的应用打下基础^[13]。

（2）指令微调（Instruction Tuning）：在预训练基础上，用带有明确任务目标的指令微调数据集对预训练模型进行训练，使模型更清楚地理解用户的指令，生成符合任务要求的输出结果^[12]。

（3）对齐优化（Alignment）：在指令微调的基础上，通过人类反馈等标注数据集等数

据内容对模型输出进行约束与优化，使其更符合实际应用需求^[12]。

经过以上三个阶段的训练之后，LLM 能够完成自然语言理解与文本生成等任务，在语义解析与内容生成方面表现出较强能力。

2.1.3 大语言模型面向智能家居场景的核心技术特性

针对传统智能家居语义控制系统在泛化能力不足、模糊指令解析精度低以及多轮交互能力有限等问题，相比传统规则匹配方法，大语言模型在语义理解与上下文处理方面具有一定优势^[14]：

（1）指令遵循能力

经过指令微调优化后的大语言模型可以较准确理解用户输入的语义目标，即使用户使用口语化表达方式，也能从其中提取出用户的控制意图，无需使用固定关键词匹配的方式。该特性可以更好适应智能家居场景下用户表达方式不同的问题^[15]。

（2）零样本 / 少样本学习能力

预训练阶段的大语言模型学习了多种不同的文本和任务表达，具有一定的泛化和迁移能力。即使没有对某个任务做专门的训练，模型也能依据已有的语义规律去完成相应的任务^[14]。智能家居控制场景中用户指令存在很大的表达差异，例如“有点冷”“调高温度”“屋里太冷了”等表达，传统的办法难以涵盖所有的表达方式，大语言模型依靠语义推理来实现统一语义理解，从而减少规则设计成本，提高系统对各种表达方式的适应性。

（3）可控的结构化生成能力

通过 Prompt 对 LLM 的输出格式与字段进行约束，生成符合要求的结构化输出^[15]。智能家居控制场景中设备可以执行的指令要包含设备标识、控制参数等核心信息，对于输出格式的规范性有较高的要求^[16]。主流结构化输出格式中 XML 格式层次表达能力强但是结构相对冗长、嵌入式设备解析成本高；简单键值对格式简洁但是可扩展性差，不能适应多设备协同、多参数控制的复杂场景；与 XML 等格式相比，JSON 语法更加简洁，解析难度小，更适合 ESP32、STM32 等边缘设备解析，在物联网设备通信场景中应用较广泛。因此在智能家居场景中，通常通过 Prompt 对输出格式进行约束，使 LLM 生成标准化 JSON 数据，以供后续设备解析与执行。

2.2 智能家居关键技术

智能家居系统主要涉及设备通信与管理、语音交互等关键技术。设备通信与管理主要是实现云端和边缘设备之间数据传输以及指令交互，对系统的实时性与稳定性具有重要影

响；语音交互技术完成用户输入获取的功能。两种技术相互配合，共同保障智能家居系统的正常运行。

2.2.1 设备通信与管理技术

智能家居系统中设备之间数据传输、状态更新、控制指令下发等都需要通过通信协议实现。通信方式设计会影响系统的响应速度、运行稳定性和可接入设备的数量。由于智能家居场景中设备数量多，部分设备硬件资源有限，通信协议需具备较低带宽占用、较快响应速度，并且能适应多设备接入、网络状态不稳定等状况。目前智能家居领域常用的通信协议有 HTTP 和 MQTT 两种，两种协议在通信方式、系统架构、应用特点等各方面都存在一定的差异，适合于不同的应用场景。

为了直观地比较两种协议在智能家居场景中适用性，本小节从核心性能入手，给出协议性能对比表，如表 2.1 所示。

表 2.1 HTTP 与 MQTT 协议核心性能及场景适配性对比

对比维度	HTTP 协议	MQTT 协议
通信模式	请求-响应式，点对点通信	发布-订阅式，代理中转通信
连接机制	短连接为主	支持长连接保活
报文开销	高，完整 HTTP 头部冗余度大	低，最小固定头部仅 2 字节
实时性	一般	高，消息实时分发
多设备并发	点对点模式 难以适配多节点同步控制	单主题支持 多设备订阅与消息分发
弱网环境	网络波动易导致请求失败， 重传开销大	支持断网重连、消息缓存 与离线消息
场景适配性	适配单设备状态查询	适配设备实时控制、 多设备协同、状态同步

HTTP 协议属于典型的请求响应式通信协议，基于客户端-服务端架构，客户端主动发起请求，服务端接收到请求后进行处理并给出响应结果^[17]。该协议实现逻辑简单，具有成熟的技术生态，可以和 Web 服务、云平台对接，在早期的智能家居系统中被广泛用于设备状态查询、单条控制指令下发等场景^[18]。但是 HTTP 协议在多设备实时控制的环境下还存在着一些不足。通信过程中频繁建立与释放 TCP 连接；即使采用长连接机制，仍需传输完整请求报文头，所以通信冗余度高、带宽占用大。点对点模式难以满足多设备并发控制的需求，在接入设备数量增多时易出现响应延迟增加，在需要多设备协同控制的情况下难以达到较高的实时性要求^[19]。

MQTT 是一种面向物联网低带宽、弱网络环境设计的轻量级消息传输协议，使用发布-订阅通信方式，用消息代理（Broker）来实现发布端和订阅端的解耦通信^[20]。MQTT 不

同于 HTTP 协议中的点对点通信方式，设备不需要直接建立连接，只需要订阅某个主题就可以实时收到消息。消息发布后，所有订阅该主题的设备都会收到相关内容，因此更适合智能家居中多设备通信和控制的场景。MQTT 协议报文小，最小固定报文头部只有 2 字节，在通信过程中带宽占用低。支持长连接机制以及三级 QoS（Quality of Service）服务质量等级，在网络条件较差的情况下仍能保持较稳定的通信^[21]。

综合来看，MQTT 比 HTTP 更适合低带宽、低延迟、多设备协同的场景，可以被广泛应用在智能家居、物联网系统中。

2.2.2 语音交互技术（ASR/TTS）

语音交互是智能家居场景里用户最常使用的一种自然交互方式，是智能家居系统中重要的人机交互方式，主要实现语音和文本之间的转换，给后续的 LLM 语义解析提供输入基础，向用户反馈系统执行的结果^[22]。语音交互技术主要包括自动语音识别（Automatic Speech Recognition, ASR）与语音合成（Text To Speech, TTS）两大部分构成，二者与 LLM 语义解析模块协同工作，共同完成语音输入与语音反馈功能。

自动语音识别模块主要完成用户输入语音信号到文本信息的转换，是系统获取用户控制指令的第一步，识别结果会直接影响到后续语义解析的效果^[23]。目前主流 ASR 技术基于深度学习框架实现，对大量的语音语料和文本数据进行训练，适应不同的用户语速、口音、发音习惯，采用降噪算法消除家居环境中产生的背景噪声干扰，在日常家居环境中可以得到较高的中文语音识别准确率。相比于早期依靠模板匹配、隐马尔可夫模型的识别方法，以深度学习为基础的 ASR 模型在泛化能力、抗干扰能力方面有明显提升，可以满足智能家居开放环境下的语音交互需求^[23]。

语音合成模块把系统发出的文本反馈信息转变成自然的语音信号，向用户反馈指令执行结果、异常提醒、状态查询等消息，属于改善人机交互体验的关键部分^[24]。当前主流的 TTS 技术已经从早期的拼接合成、参数合成，转向基于深度学习的端到端语音合成方案，能够生成较为自然的语音反馈^[25]。

综合来看，MQTT 协议在多设备通信场景下具有较好的实时性与稳定性，而 ASR/TTS 技术则能够提升语音交互体验。这些技术共同构成了智能家居系统实现的重要基础。

第三章 系统需求分析

本章结合智能家居自然语言交互场景，对系统功能需求、扩展需求、可靠性需求以及实现可行性进行分析，为后续系统设计提供依据。

3.1 功能需求分析

3.1.1 自然语言模糊指令解析功能

智能家居场景中用户输入的控制语句通常较为口语化，且存在表达不完整、语义不清等情况，需要系统有较强的自然语言理解能力。例如，当用户说“太暗”时，系统不仅需要理解用户需求，还应自动执行相应的设备控制操作（打开灯）。

为了实现自然语言到设备控制指令的转换，系统需要对用户输入内容进行语义分析，主要包括以下几个部分：

意图识别（Intent Recognition），其主要用于识别用户期望系统执行的操作。传统的算法一般采用关键词匹配的方式，当用户表达方式发生改变时，容易出现识别不准的现象^[26]。相较于基于规则的方法，大语言模型的语义理解方法是把上下文信息纳入建模范围，从而对用户的输入作出整体分析，识别用户的真实意图，再将其映射到相应的控制操作上。

实体信息提取（Entity Extraction），是在确定用户意图之后，系统需进一步提取具体控制对象及相关参数信息，如设备类型、温度等控制参数^[27]。该过程直接影响后续设备控制的准确性。以“空调调低一点”为例，系统要同时识别出设备对象“空调”、控制动作“调低”这两个关键信息。

结构化指令生成（Structured Output Generation），主要用于将语义解析结果转换为标准化数据格式，供后续设备解析执行^[28]。本文采用 JSON 作为控制指令的输出格式，用结构化数据来描述设备名称、控制动作和参数信息，方便系统内部数据传输以及功能扩展。

3.1.2 上下文感知多轮对话功能

在智能家居交互过程中，用户通常不会一次性完整表达所有控制需求，而是基于前序对话逐步完成控制操作。因此系统不能只对当前的输入进行处理，还要结合之前的对话信息来做出语义判断，才能实现多轮连续的交互^[29]。例如，用户在上一轮已经指定“空调”后，下一轮可能只说“再冷一点”，此时系统需要根据前文内容判断当前控制对象。

为了实现上述功能，在语义解析的过程中引入上下文辅助。系统处理当前用户输入时，会将一定范围内的历史对话内容以及当前指令一同输入模型，加强模型对连续语义的理解

能力^[30]。同时为了避免历史信息过多造成语义混乱，系统同时采用时间范围的上下文管理机制。当历史对话超过设定时间阈值之后，相关的内容就会被自动清除，减少历史信息对当前控制结果的影响。

利用上下文信息，系统可以一定程度上保持对话内容的连续性，减少用户重复输入设备名称或控制对象的情况，从而提升多轮交互体验。当前系统默认保留最近 5 分钟内的对话内容，可以支持常见的连续控制操作。

3.1.3 语音交互功能

为了提高智能家居系统的使用便利性，系统需要具备语音交互能力。用户在实际使用过程中，很多时候并不方便通过手机或按键输入指令，因此可以直接通过语音与系统进行交互，完成设备控制和信息查询等操作。

系统首先需要完成语音采集，并将采集到的语音数据转换为文本信息。由于家庭环境中可能存在电视声音、人员交流声等干扰因素，因此语音识别过程需要具备一定的抗干扰能力，以保证识别结果的准确性。

语音转换为文本后，系统需要对文本内容进行分析，判断用户想要执行的具体操作。例如用户说“打开客厅灯”，系统需要识别出控制对象为客厅灯，控制行为为打开，并生成对应的控制指令。如果用户输入的是普通问题，如天气查询或简单问答，系统则返回相应的文本结果。

为了增强交互体验，系统还需要支持语音播报功能。当设备控制完成后，系统能够将执行结果转换为语音进行反馈，让用户及时了解控制情况。通过语音输入与语音反馈相结合，可以减少用户操作步骤，提高系统使用的便捷性。

3.2 可扩展性需求分析

为了满足不同设备接入、功能扩展等各方面的需要，在设计阶段就应具备良好的可扩展性，从硬件接口和软件架构两个方面进行设计。

在硬件层面，系统基于 ESP32 与 STM32 提供的 GPIO、SPI、I2C 等接口完成外设连接。通过统一接口与通信协议，可实现新硬件设备的快速接入。传感器设备根据通信方式选择对应接口的连接，用于实现数据采集与控制信号传输。

在软件层面，对各种设备进行统一的封装处理，使用比较通用的设备驱动结构。驱动模块根据设备类型及通信协议完成初始化、数据读取与控制处理等操作，新增加的设备只需进行基本参数配置即可完成接入。

系统整体采用模块化设计，把语义解析、设备通信、控制执行等功能划分成相互独立的模块，用统一接口完成数据交互。该设计可降低模块间耦合度。在后续的功能扩展或模块调整时，只需对相应部分进行修改，不会对系统整体运行造成较大影响。

3.3 可靠性需求分析

智能家居系统在运行过程中涉及设备控制、网络通信以及数据传输等多个环节，因此系统需要具备较好的可靠性，保证各项功能能够正常运行。

在网络通信方面，ESP32 需要通过 Wi-Fi 连接服务器完成数据交互。当网络出现波动或连接中断时，系统应能够自动重新连接服务器，避免因通信异常导致控制指令无法发送或接收。

在设备控制方面，系统需要保证控制指令能够正确传递给目标设备。当出现设备离线、通信失败等情况时，系统应给出相应提示，方便用户了解当前设备状态。对于已经执行完成的控制操作，系统也需要及时返回执行结果。

另外，设备运行状态需要定期上传到服务器，包括设备在线情况、网络连接状态等信息。通过状态监测可以及时发现设备运行过程中存在的问题，提高系统运行的稳定性。

系统采用模块化结构设计，语义解析、设备通信和硬件控制等功能相互独立。当某一模块出现异常时，可以减少对其他模块的影响，便于后续维护和功能扩展。

3.4 需求可行性分析

从技术实现角度来看，本文所设计的智能家居系统具有较好的可行性，主要体现在语义处理能力、硬件支持及系统架构设计等方面。

语义处理部分，当前大语言模型已具备自然语言理解、意图识别及控制指令生成能力，能较处理好智能家居场景中的口语化表达。结合了 Prompt 约束和规则辅助之后，系统可以使得指令解析更加准确，基本满足日常家居控制场景下的使用需求^[31]。

硬件上 ESP32 主要负责网络通信和云端数据交互，STM32 负责底层设备驱动和控制执行。两类芯片分工明确，但可协同完成系统通信与控制任务，从而形成完整的设备控制流程，给系统运行提供基本的硬件支撑。

系统架构采用云端与边缘协同的工作模式，将语义解析等计算密集型任务部署于云端处理，将设备控制任务下沉至本地执行。这样可以降低嵌入式设备的运算负担，从而提升系统响应速度与稳定性。

目前系统所涉及的大部分关键技术已经比较成熟，根据现有的硬件和软件条件，具有一定的实现基础和工程应用可行性。本章主要对系统功能需求、可扩展性需求、方案可行性进行分析，后续章节将在此基础上进一步完成系统架构设计与功能实现。

第四章 系统设计

4.1 系统架构设计

4.1.1 分层架构总体设计

本文采用“五层三端”的整体架构实现智能家居控制功能。系统从功能上划分为用户交互层、LLM 推理层、数据存储层、设备管理与驱动层以及硬件执行层；从部署角度来看，则包括用户端、云端与设备端三个部分。整体结构如图 4.1 所示：



图 4.1 五层三端结构示意图

云端主要负责大语言模型语义解析、上下文推理以及控制指令生成等计算任务，设备端完成底层硬件控制及设备执行，用户端接收用户的输入并返回控制结果。将系统功能进行划分，可以保证基本控制效率的同时，减少模块之间相互影响，便于以后功能的调整和扩展。

在整体结构中，各个层之间使用统一接口进行数据传输，各个模块相对独立，后续功能修改时不需要对整个系统做大的改动。

系统运行时，用户输入首先经过交互层处理，然后传送到 LLM 推理层进行语义解析。LLM 推理层完成语义解析后，将结构化控制指令发送至设备管理层，随后由设备管理层下发至硬件执行层完成设备控制。控制执行单元将控制执行结果传送给用户，最终完成整个智能家居控制流程。

4.1.2 用户交互层设计

用户交互层主要是接收用户的输入，然后返回系统控制的结果，是用户和系统交互的重要部分。由于智能家居场景下语音操作更符合日常使用习惯，因此本系统主要采用语音交互。

语音输入时系统先用 ASR 模块把用户的语音内容转换成文本信息。实际使用环境中会有噪声，系统会对原始语音数据做基本的预处理以及简单的降噪，从而减小环境噪声对识别结果的影响。语音识别之后，系统会对文本内容进行基础整理，即语句分割、标点补充等，方便之后语义解析模块的处理。

语音反馈阶段系统用 TTS 模块把文本信息转换成语音输出。根据不同的反馈内容，系统能生成对应的语音播报信息，从而提升交互自然性。

交互层会对于一些不完整的输入做简单的处理。当用户输入存在指代不明确或语义不完整的情况时，系统会结合最近一次的操作记录进行简单推理。

4.1.3 云端大语言模型推理层设计

大语言模型推理层就是将用户输入理解成结构化的指令，然后给后面的设备控制提供指导。该层主要包括意图解析与上下文管理两个模块。该层采用云端大语言模型接口调用方式实现语义推理功能，主要包括接口通信、意图解析以及上下文管理等模块。

系统在接收到用户输入后，首先对文本内容进行预处理，并结合预定义 Prompt 模板构造请求数据。随后通过接口向云端大语言模型发送推理请求，由模型完成自然语言语义理解与控制意图分析。系统采用 JSON 格式作为统一的数据交互方式，模型返回结果经过解析后进入后续设备控制流程。

意图解析模块主要是将用户当前的控制需求转化为系统可以识别的结构化控制信息，即把用户的自然语言输入转换成系统的控制指令。系统利用大语言模型的语义理解能力，对用户的输入内容进行分析。随后通过 Prompt 对模型输出格式进行约束，并使用预定义 JSON 字段模板规范输出结构。同时结合关键词规则对生成结果进行合法性校验，从而使生成结果能够直接用于后续设备控制流程。输出结果中会给出设备名称、控制动作以及参数等内容。

为了方便后面模块对模型输出结果进行处理，系统对返回的数据进行了统一的格式设计。生成结果主要包括设备对象、控制行为以及参数信息等字段，以固定的格式返回。通过限制输出格式，可以在一定程度上降低自然语言生成结果的不确定性，从而提高控制指令解析的稳定性。

上下文管理模块一般用在多轮交互场景中。系统会保存最近一段时间的交互记录，即当前控制设备及有关状态信息，之后再解析时会用到这些信息作为辅助输入参与模型推理。

加入上下文信息之后，系统对于部分表达不完整的控制语句也能做出比较准确的语义判断。用户先输入打开空调，再输入凉快一点的时候，系统可以根据前面的对话内容判断

当前调节对象仍然是空调。

相比传统关键词或者规则匹配的方式来说，大语言模型对于口语化表达、语义模糊、连续对话等情况有着较好的适应性，在比较复杂的人机交互过程中能够更好地理解用户的控制意图。

4.1.4 设备管理与驱动层设计

设备管理与驱动层主要是完成控制指令的下发、设备的通信和硬件驱动，是实现设备联动与通信控制的核心部分。

由于各个智能设备所用的通信协议不同，因此系统对通信过程做了统一封装。目前系统主要支持 MQTT、HTTP 两种通信方式，预留了其他的协议扩展接口。

当系统接入新的设备时，只需要增加对应的协议适配模块就可以完成通信连接，不需要对整个架构进行修改。设备控制的时候，LLM 推理层产生的结构化控制数据先转换成相应的通信消息，再发给目标设备执行。

设备返回的状态信息会经过统一处理后上传至系统，供后面的状态更新以及数据处理使用。设备管理模块主要是对当前设备的状态进行维护，对控制任务进行分发。当设备不能执行控制指令时，系统会执行有限次数的重试机制，并返回对应异常状态信息。

4.1.5 硬件执行层设计

硬件执行层主要是完成设备控制指令的执行和底层硬件驱动，是系统实现智能家居控制的重要部分。

为了同时满足网络通信和底层实时控制的要求，系统使用 ESP32 和 STM32 两个控制器配合工作的双控制器结构。ESP32 主要完成 WiFi 网络连接、MQTT 消息接收、与云端数据通信；STM32 完成底层外设驱动以及具体设备的控制。两个控制器之间通过 UART 串口进行数据通信。ESP32 接收到云端发出的控制指令后，会将结构化的控制数据通过串口发送给 STM32，再由 STM32 完成对应的硬件设备控制。

系统会检测设备运行状态，把检测结果传送到上层模块，供上层模块用作状态更新、异常处理等用途。

相比于单控制器方案，ESP32 和 STM32 分工协作的方式可以减少网络通信任务对底层控制过程的影响，提高系统运行的稳定性与响应效率。

4.1.6 数据存储层设计

系统运行时产生的设备状态、用户交互记录、日志信息等全部由数据存储层统一管理。考虑到嵌入式设备资源有限，用轻量化的存储方式实现数据的管理。

系统所存数据主要是设备信息以及用户交互记录。设备信息有设备类型、通信方式、运行状态等信息，可以进行设备管理、状态查询等。交互记录就是用户输入的内容，系统响应的结果以及时间信息等数据的储存。

4.1.7 系统数据流与控制流程设计

为了说明系统各个模块之间数据的交互关系，本文对整个控制流程做了统一的设计。系统运行的时候，数据会依次通过用户输入、语义解析、设备控制、状态反馈等环节来完成一个闭环的控制过程。

整个系统的工作流程如图 4.2 所示：

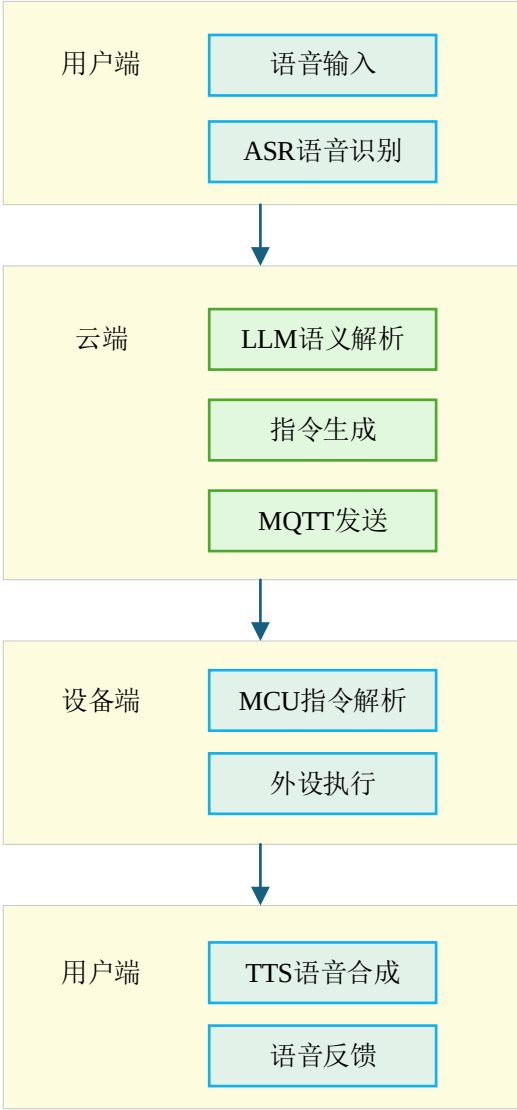


图 4.2 系统工作流程示意图

交互层中用户可以用语音方式发出控制指令。系统先用 ASR 模块将语音识别成文本信息。系统之后会对识别结果进行基础预处理，包括无效字符过滤等操作来提高后续语义解析的准确性。处理过的文本数据再传递给语义理解模块。

在云端，文本信息传递给 LLM 推理层，系统结合意图解析结果与上下文信息生成结构化控制指令。为了保证系统稳定运行，在控制指令进入设备执行阶段之前还要对控制指令的合法性、格式进行校验，防止出现异常数据或者错误指令直接作用于硬件设备。

控制指令发出之后，就会通过通信模块传送到 ESP32。ESP32 接收到来自 MQTT 的消息之后，会通过 UART 串口将控制数据传送给 STM32，STM32 对接收到的控制数据进行处理，然后执行灯光开关、设备状态调节等具体的操作。

设备执行完毕后，相关的状态信息就会被上传到服务端，并且存入数据存储层中。系统会对执行结果作出相应的反馈信息，用语音形式反馈给用户。

经过以上过程，就完成了由自然语言输入到实际设备控制再到语音反馈的全过程，实现了基础智能家居控制功能。

4.2 核心模块详细设计

为进一步说明系统内部各核心模块的运行机制，以下分别对意图解析模块、安全机制与异常指令拦截模块、设备控制执行模块、MQTT 通信机制模块设计进行详细论述。

4.2.1 基于大语言模型 API 的意图解析模块设计

本系统的大语言模型能力主要通过接入小智平台实现。ESP32 设备完成语音采集之后，首先由语音识别模块将用户语音转换为文本内容，再由小智平台统一调用云端大语言模型 API 进行语义解析。

系统运行过程中，小智平台会根据用户输入内容以及预设 Prompt 模板构造请求数据，并发送至大语言模型接口。模型完成推理后返回结构化 JSON 数据，平台对结果进行解析，并将设备名称、控制动作以及参数信息发送给 ESP32 控制端执行。

基于大语言模型 API 的意图解析模块把用户的自然语言转换成系统可以识别的控制指令，是整个语义处理过程中的关键部分。模块通过调用云端大语言模型接口实现语义理解能力，并结合规则约束与结构化输出机制，提高指令解析的准确性与稳定性。

智能家居应用场景中用户的输入带有很强的口语化特征。以“有点热”“帮我把灯打开”等表达为例，缺少具体的数值参数，部分语句还有语义不完整、描述模糊等问题。为此，系统采用大语言模型加规则匹配的方式进行语义解析，以提高指令识别准确率。

整个模块主要包括输入预处理、语义解析、结果校验这三个部分。

系统在输入预处理阶段会先对用户的输入内容做基本的整理，包括去掉没有实际意义的语气词，处理部分口语化的表达等。再利用关键词匹配的方式来提取设备名称、控制动

作等，为模型推理提供参考信息，减少模型输出偏差。

为了统一系统支持的设备类型与操作方式，系统预先构建了智能家居设备操作指令集，对设备对象、操作行为以及参数类型进行约束，如表 4.1 所示。

语义理解阶段系统会预先定义好大语言模型的输出格式，保证模型返回的解析结果按规定的格式呈现出来。解析结果主要是设备类型、控制动作、参数信息等字段。用户输入“把灯打开”时，系统把“灯”当作目标设备，“打开”当作控制行为。为了方便后面设备控制和数据传输，系统统一用 JSON 格式来封装解析结果并保存。

相比自由文本输出方式，结构化生成可以减少结果不一致的现象，也便于后面程序解析和设备控制。

为了进一步提高语义解析稳定性，系统预先构建设备名称与控制行为相关词典，对用户输入内容进行关键词匹配，并将匹配结果作为辅助上下文加入 Prompt 中，使模型能够更准确地理解用户意图，减少由于语义偏差造成的解析错误。

系统输出阶段会对生成结果做基本的格式校验，即检测 JSON 结构是否完整。当检测出输出结果异常时，系统会通过规则补全或再次请求模型生成的方式进行修正。

系统会对设备名称、操作类型等重要字段进行合法性校验，防止错误的指令进入后续的设备控制阶段。

经过以上处理之后，系统就可以把自然语言输入转化为控制指令，给后面的设备执行提供数据支持。

表 4.1 智能家居设备操作指令集

操作对象	操作方式	参数说明
灯光	打开、关闭	无
空调	打开	number (默认设置 27，范围 15~35)
空调	关闭	无
空调	温度上升、温度下降	number (空调调节温度幅度)
所有电器	打开、关闭	无

4.2.2 安全机制与异常指令拦截模块设计

由于大语言模型生成的结果具有不确定性，用户输入中也会出现异常控制的内容，所以系统增加了异常指令拦截模块，防止异常指令直接进入设备控制阶段。

该模块在 LLM 意图解析之前，主要是采用关键词过滤与语义安全判定两种方式对用户输入进行检测。

(1) 关键词规则拦截机制：关键词过滤属于第一步检测方式，主要是对明显异常的输入进行识别。创建一个面向智能家居场景的关键词库，包括高危设备操作、异常参数设

置、无效控制行为等。

用户输入经过预处理之后就会进入到关键词检测模块。当输入内容命中关键词库的时候，系统就会立即停止后面的控制流程，向用户发出提示信息；如果没有检测到异常，就会继续进行后面的语义解析。

该方式实现比较简单，可以减少部分无效输入进入模型推理阶段，也可以降低后续模型推理负担。

(2) 语义安全判定机制：由于部分风险指令不会直接包含敏感关键词，所以系统又加入了大语言模型的语义安全判定机制。

系统在实际运行时会把安全约束规则加入到 **Prompt** 里，然后模型就会根据用户输入来判断是否安全。当输入含有危险控制行为、异常参数等时，系统就判定为风险指令，停止后面的流程。

只有输入内容安全检测通过之后，系统才会继续进行结构化意图解析。

相比于简单的关键词过滤来说，该方法能够识别部分隐式风险表达，进而提高异常输入识别的能力。

(3) 拦截流程设计：整个异常指令拦截模块被放置在系统控制流程前端。用户输入后，首先要经过关键词检测和语义安全判定，只有两部分检测都通过之后，系统才会继续执行后面的设备控制操作。

系统会对拦截的结果做日志记录，以后作为关键词库更新、**Prompt** 优化的依据。

另外为了提高系统的实时性，关键词匹配部分采用简单的字符串匹配方式，语义安全判定和意图解析尽量通过一次模型调用来完成，减少额外的延迟。

4.2.3 设备控制执行模块设计

设备控制执行模块将语义解析之后的结构化指令转换成实际的硬件控制操作，是系统实现设备联动的重要部分。

系统用 ESP32 和 STM32 协同完成控制任务。ESP32 负责网络通信以及消息转发工作，STM32 做底层设备驱动以及具体控制执行。ESP32 在运行时通过 WiFi 接入网络，和 MQTT 以及服务端建立通信连接。系统发出控制指令之后，ESP32 就会收到对应的 MQTT 消息。然后 ESP32 解析出 JSON 数据，把内容转换成适合串口通信的数据格式，通过 UART 发送给 STM32。

STM32 接收到串口数据之后，就会根据指令的内容来执行相应的硬件操作。系统主要用 GPIO 和 PWM 接口来完成设备的控制，即灯光开关控制、继电器驱动、设备状态调节

等。由于 STM32 主要做底层控制，因此可以较快地响应设备操作请求，提高控制过程的稳定性。

ESP32 和 STM32 之间通信使用相同的数据帧格式来传输数据。数据内容主要是指令类型、设备信息、校验字段等，保证通信数据的完整性。ESP32 在发送数据之前会完成数据的封装，STM32 在接收到数据之后会对数据进行解析、校验，最后返回设备的状态信息，起到基本的状态反馈作用。把网络通信与设备控制分到不同的控制器上，可减小模块间的相互干扰。后续如果需要增加新的设备类型，只需要在 STM32 侧补充对应的驱动程序即可，不需要对网络通信部分做大的修改。

系统接收到“打开灯光”的控制指令之后，ESP32 就会对消息进行接收和转发，STM32 控制 GPIO 输出高电平，实现点亮灯的操作。

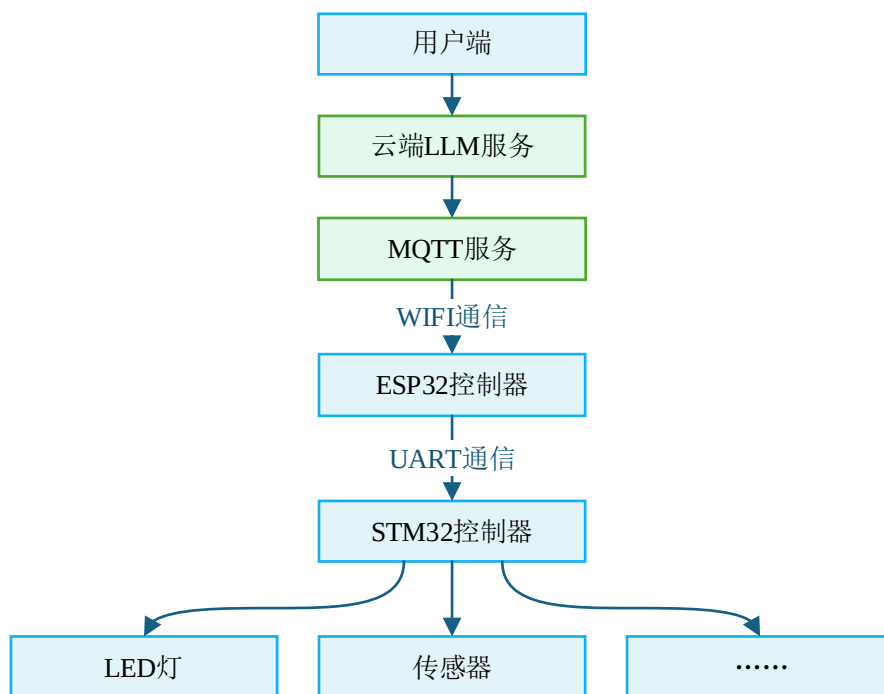


图 4.3 ESP32 与 STM32 协同控制架构图

4.2.4 MQTT 通信机制模块设计

为了实现云端与设备端之间的数据通信，系统采用 MQTT 协议作为主要通信方式。

在系统架构中，MQTT 通信模块主要负责语义解析结果的传输以及设备状态信息的同步。系统通过 MQTT Broker 对消息进行统一管理，设备端通过订阅指定 Topic 接收控制指令，云端则通过发布消息完成设备控制，从而实现云端与设备端之间的数据交互。

为了便于数据解析与后续扩展，系统统一采用 JSON 格式封装 MQTT 消息内容。消息数据主要包括设备名称、控制动作以及参数信息等内容。

在 Topic 设计方面，系统根据不同功能对消息主题进行分类管理。控制类主要用于设

备控制指令传输，状态类主要用于设备运行状态同步。通过不同类别的划分，可以降低不同类型消息之间的相互影响，提高系统通信管理的清晰性。

为了保证通信过程的稳定性，系统在 MQTT 通信过程中增加了连接状态检测机制。当网络出现异常或者 MQTT 连接中断时，系统能够重新建立通信连接，减少由于网络波动造成的设备离线问题。

此外，为了避免异常数据影响设备运行，系统在接收到 MQTT 消息后，会对 JSON 数据结构以及关键字段进行基础校验。当消息格式异常或者字段不合法时，系统会放弃执行对应控制操作，从而提高系统运行安全性。

通过 MQTT 通信机制，系统实现了云端语义解析结果与设备控制模块之间的数据交互，为智能家居设备控制与状态同步提供了通信基础。

第五章 系统实现

5.1 系统硬件实现

5.1.1 系统整体硬件架构设计

系统硬件部分使用 ESP32 和 STM32 两个控制器配合起来完成整个控制功能。ESP32 主要实现无线网络通信、MQTT 消息交互、云端控制指令转发；STM32 主要负责底层外设驱动、设备状态控制、传感器数据采集。两者之间采用 UART 串口来传递数据。双控制器分工设计可以减小网络通信对底层实时控制的影响，有利于以后增加新的设备模块，提高系统的整体扩展性。

整体结构如图 5.1 所示。

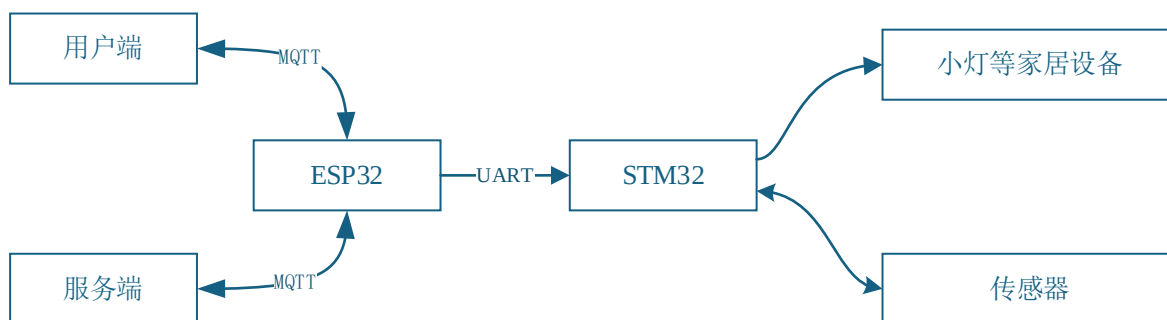


图 5.1 系统整体架构图

5.1.2 硬件连接与实物实现

系统硬件连接采用模块化结构设计，由 ESP32 主控模块、STM32 控制核心、传感器模块以及执行设备组成。各模块之间通过 UART 串口与 GPIO 信号进行连接，实现数据采集与设备控制的统一管理。系统实物连接如图 5.2 所示。

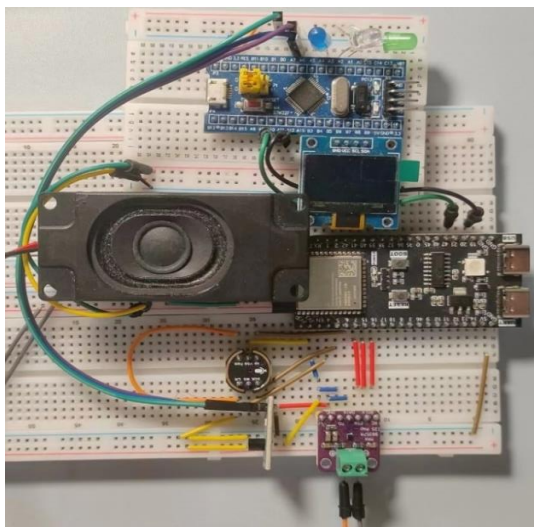


图 5.2 实物连接图

5.1.3 ESP32 模块

ESP32 在系统中主要负责网络通信功能,即连接局域网并和 MQTT 服务器建立通信。与传统的单片机外接 WiFi 模块方式相比,ESP32 芯片集成了无线网络功能,减少了外围硬件模块的数量,从而使得整个硬件结构更加简单。

ESP32 模块实物如图 5.3 所示。

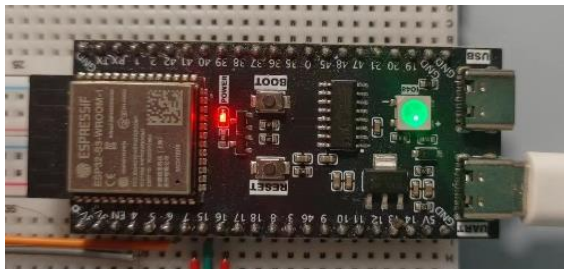


图 5.3 ESP32 开发板实物图

ESP32 上电后首先完成 WiFi 初始化,然后做预设网络连接。之后会与 MQTT 服务器建立连接,并订阅对应的主题。采用 MQTT 发布/订阅机制,实现云端与终端设备之间的数据交互。

当用户发出自然语言控制请求之后,服务端就会进行语义解析,得到相应的结构化控制数据,再通过 MQTT 将控制消息发给 ESP32。ESP32 接收到消息之后,会对其内容进行解析,从中提取出设备类型、控制动作、参数信息等信息,再把它们转变成 STM32 可以识别的串口控制指令,经由 UART 接口发出执行。

5.1.4 STM32 模块

STM32 是系统的底层控制模块,主要是完成设备驱动、GPIO 控制、传感器数据采集等工作。系统用 STM32 控制 LED 灯等执行设备,对环境温度数据进行读取和处理。

STM32 模块实物如图 5.4 所示。

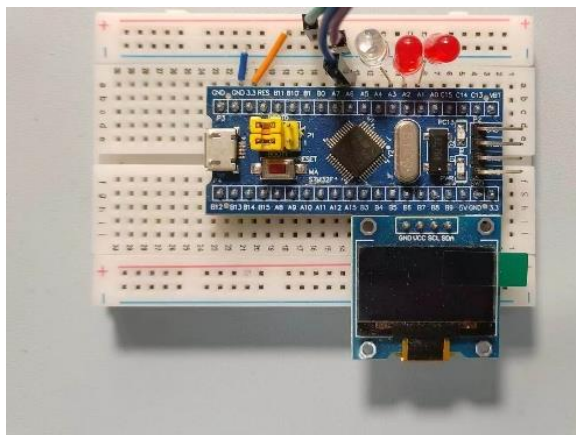


图 5.4 STM32 开发板实物图

当 STM32 接收到串口控制指令之后,就会先对指令内容进行解析,再根据不同的控

制类型执行相应的硬件操作。用户发出打开灯光指令时，STM32 控制对应的 GPIO 输出电平来使 LED 点亮，接收到关闭指令时，关闭对应的 GPIO 输出以达到关闭设备的目的。

除了设备控制外，STM32 还需要对温度传感器数据进行采集，把当前环境温度信息与收到的控制指令结合起来，对控制指令进行进一步判断。

由于 STM32 有较好的实时控制能力，可以比较稳定地完成底层设备控制以及外设管理的任务。同时把设备控制部分独立部署到 STM32 侧，有利于以后增加其他外设模块。

5.1.5 传感器与执行设备

为了检验系统的功能，本文接入了温度传感器、LED 灯等设备，作为执行设备。温度传感器采集环境温度数据，传送给 STM32 进行处理；LED 灯模拟智能照明设备情况，用 GPIO 引脚来实现灯光开关控制。采用上述设备可以对环境感知、设备控制、状态反馈等进行验证。

各个模块之间使用杜邦线、跳线连接。原型设计结构简单，方便之后系统调试和功能扩展，如图 5.5 和图 5.6 所示

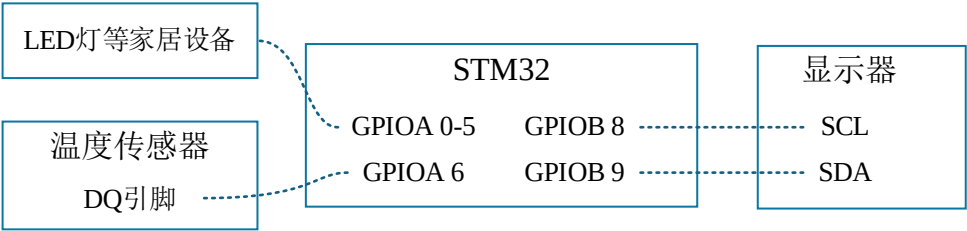


图 5.5 传感器、执行设备与 STM32 连接示意图

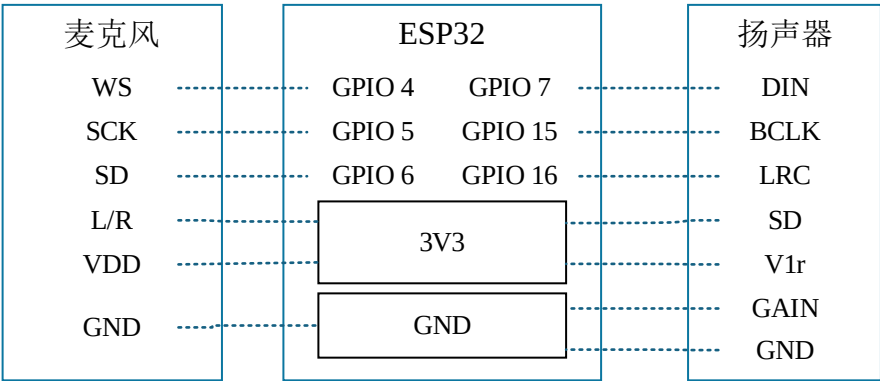


图 5.6 麦克风、扬声器与 ESP32 连接示意图

5.1.6 ESP32 与 STM32 通信方式

ESP32 与 STM32 之间采用 UART 串口通信方式进行数据传输，通信波特率设置为 115200。系统运行时，ESP32 收到云端控制消息之后，将控制内容转化为串口指令发送给 STM32；STM32 完成设备控制之后，再把设备状态信息反馈给 ESP32，由 ESP32 上传到服务端。

在硬件连接上，ESP32 的 TX 引脚和 STM32 的 RX 引脚相连，ESP32 的 RX 引脚和 STM32 的 TX 引脚相连，两模块共地。

该通信方式可以满足系统的数据传输要求，硬件连接和程序实现都比较简单，适合本系统嵌入式控制场景。

ESP32 与 STM32 通信接线示意图，如图 5.7 所示。

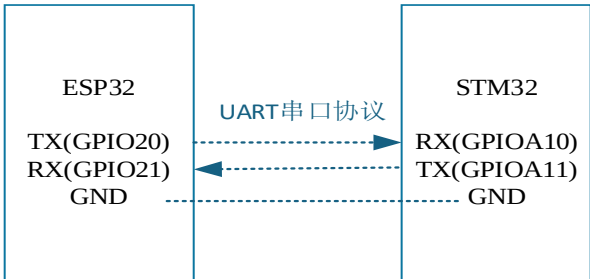


图 5.7 ESP32 与 STM32 通信接线示意图

系统硬件平台如表 5.1 所示。

表 5.1 硬件平台配置表

硬件模块	型号	功能
主控通信模块	ESP32	WiFi 通信、MQTT 消息转发
控制核心模块	STM32	设备控制与状态反馈
温度传感器	DS18B20 测温模块	环境数据采集
执行设备	LED 灯	模拟家居设备
通信方式	UART	ESP32 与 STM32 数据通信

5.2 软件设计实现

5.2.1 意图解析功能实现

‘基于云端大模型 API 的意图解析模块主要是将用户输入的自然语言转换成设备可以执行的控制指令，是整个系统实现自然语言交互的关键部分。

智能家居场景中用户的口语化程度较高，“有点热”、“帮我开灯”、“屋里有点暗”等表达中并没有明确的控制参数，传统的基于固定规则的方式很难完成语义理解。因此本文采用大语言模型和环境状态信息相结合的方式来实现用户的意图识别。系统整体意图解析流程图如下图 5.8 所示。

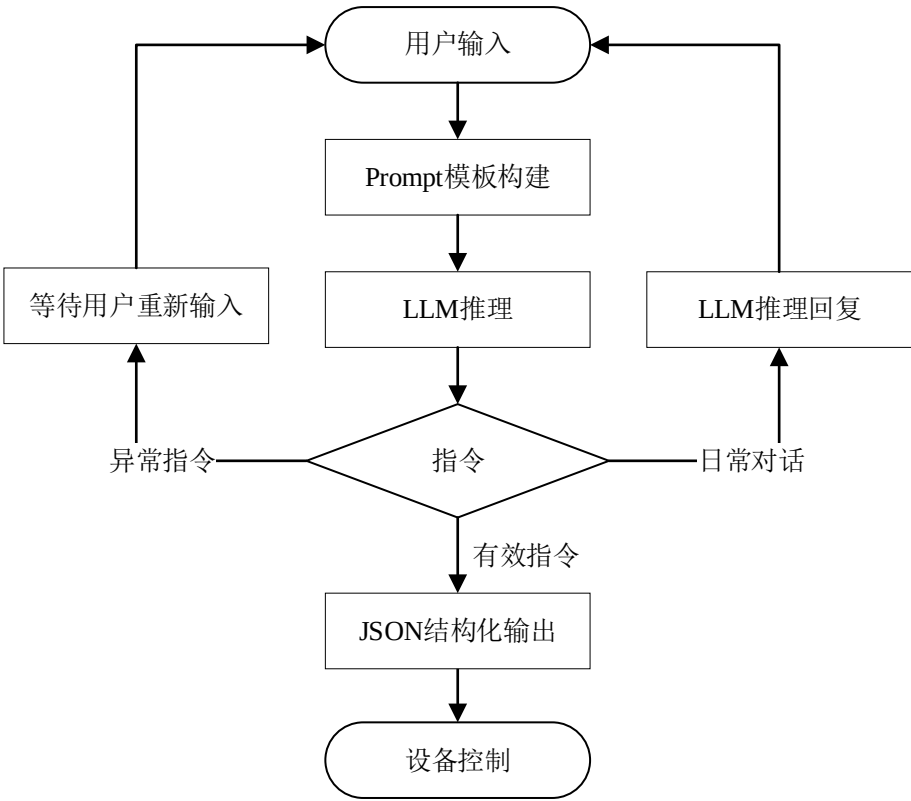


图 5.8 LLM 意图解析流程图

系统运行的时候，用户发出的语音先会被 Prompt 模板所构建，然后再经过 LLM 推理得到语义理解以及意图解析。

系统在运行过程中采用接口方式调用云端大语言模型服务。根据用户输入内容、当前设备状态信息以及预定义 Prompt 模板构造请求数据，再通过请求发送至大语言模型接口。请求数据主要包括模型名称、上下文消息以及推理参数等内容。请求数据示例如算法 5.1 所示：

算法 5.1 请求数据示例

```
{
  "model": "xxxx",
  "messages": [
    {
      "role": "system",
      "content": "你是智能家居控制助手，请返回 JSON 格式控制结果"
    },
    {
      "role": "user",
      "content": "帮我打开灯"
    }
  ]
}
```

系统完成请求发送之后，大语言模型会返回结构化 JSON 数据。系统随后对返回结果进行解析，并生成对应的设备控制结果。

用户输入“帮我打开灯”，系统就会判断出对应的设备为灯光设备，控制行为为开启操作，生成相应的控制结果。

系统不会直接依据模型返回结果控制设备，而是会联系环境状态信息加以判定。用户输入“我觉得有点冷”时，系统就会读取当前的温度传感器数据，并进行相应的模糊控制。

为了提高高频控制场景下识别的准确性，系统还加入关键词辅助匹配的机制。系统事先整理出常见的设备名称及控制行为相关的词汇，即打开、关闭、灯光、空调等。当输入内容中出现对应的关键词时，系统就会根据关键词结果对设备类型和控制动作进行辅助判断，以减少复杂语句、口语化表达以及语义不完整情况下的解析误差。

经过多次的功能测试之后，系统可以比较稳定的完成从自然语言到设备控制指令的转换，对于一些模糊的表达、日常口语化的输入有较好的适应性，可以满足基本的智能家居控制场景的使用需求。

5.2.2 安全校验与异常指令拦截实现

为了避免异常的控制指令直接进入设备执行阶段，系统加装了安全校验模块，对危险的操作以及异常的输入进行检测并加以阻止。

整个安全校验过程由关键词过滤和语义安全判定两部分组成。整体安全校验流程图见图 5.9。

用户输入控制指令后，系统就会执行关键词检测。系统预先建立敏感关键词库，危险操作、异常参数等都在其中。检测到相关敏感词时，系统就会立即终止后面的控制流程，给用户发出提示信息。

由于部分风险表达中没有包含敏感关键词，所以系统又加入了大语言模型语义安全判定机制。系统在实现时会把安全约束规则加入到 Prompt 里，模型先对用户的输入内容做安全性判断，然后才开始生成控制指令。

如果模型认为当前输入有潜在风险，则不会执行后面的设备控制，只有安全校验通过之后才开始正常的语义解析和设备控制。

在所有的控制链路上，两级安全校验机制都处于设备执行之前，因此可以减少异常指令进入底层控制模块的情况，提高整个系统的安全性。

在测试过程中构造出多组危险指令以及异常输入来检验。测试结果表明大部分异常输入在关键词检测或者语义判定阶段可以被拦截，设备端没有出现异常指令误执行的情况，

系统运行比较稳定。

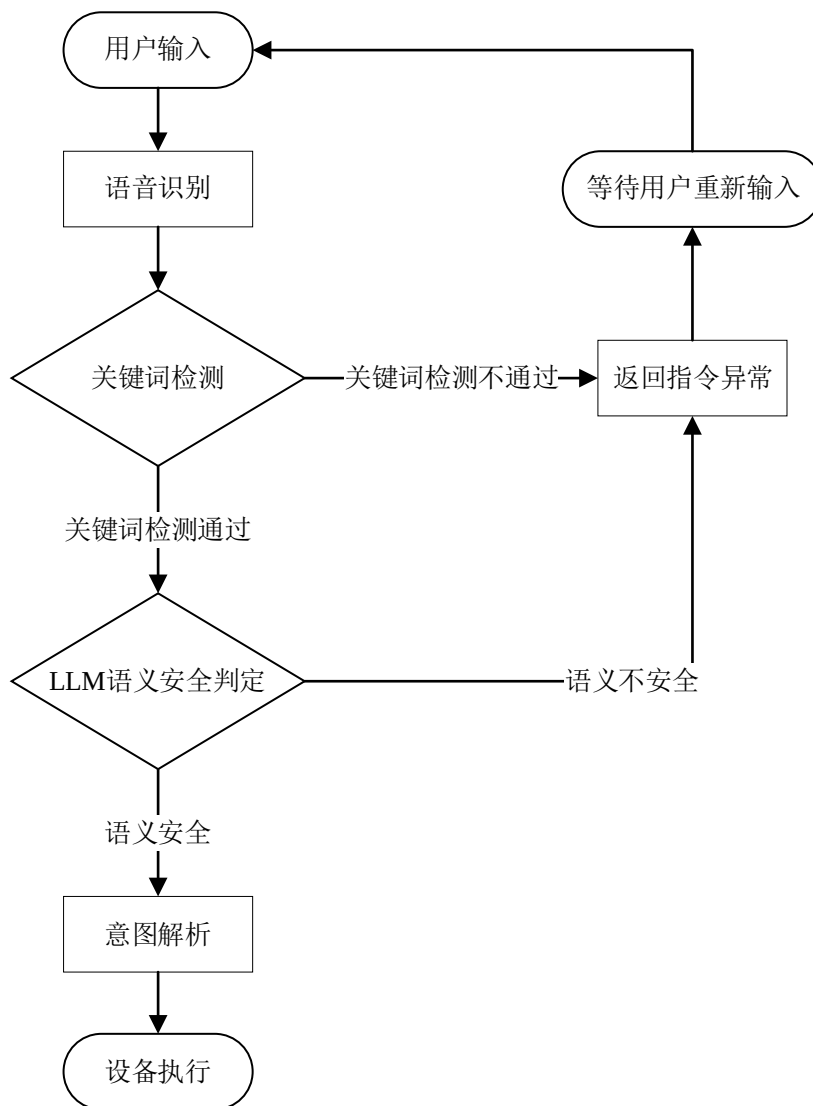


图 5.9 安全校验流程图

5.2.3 语音交互功能实现

为了改善系统交互体验，在智能家居控制系统中加入语音交互功能，完成了语音输入、语义解析、语音反馈的全过程。

语音交互模块包含语音识别（ASR）、大语言模型语义解析、语音合成（TTS）这三个部分。语音交互流程图如下图 5.10 所示。

语音输入阶段，系统会使用麦克风来采集用户的语音，之后再调用语音识别接口将语音内容转成文本信息。为了减少环境噪声对识别结果的影响，在音频上传之前对音频做简单的降噪和静音段处理，从而提高语音识别的准确率。

识别出的文本信息会被发送到服务端，大语言模型对文本信息进行语义解析之后就会产生对应的设备控制指令。系统之后将控制结果发送给 ESP32，由 STM32 进行具体的设

备控制操作。

设备控制完成后，系统就会根据当前的执行结果产生相应的文本反馈信息，比如灯已经打开、空调温度为 26 摄氏度等，然后通过语音合成接口生成语音反馈，最后由音频设备播放给用户。

实际使用中系统加入了语音合成模型来完成语音播报。根据不同的设备状态以及控制结果，系统会对提示语内容做简单的区分，从而提高整个交互体验。

整个语音交互过程当中，用户不用手动输入控制命令，只要说出自然语言就能对设备进行控制或者查询状态，提高了智能家居系统的人机交互便利性。

此外，系统不仅能够完成设备控制指令解析，还能够基于大语言模型实现基础自然对话功能。在非控制类输入场景下，系统可根据用户输入生成对应的自然语言回复，实现简单的人机交流。

例如，当用户输入“今天天气怎么样”“你是谁”“播报一条新闻”等非设备控制内容时，系统会调用大语言模型生成文本回复，并通过 TTS 模块进行语音播报。

该功能使系统不仅具备设备控制能力，同时具备一定的智能陪伴与自然交互能力，提高了整体人机交互体验，也进一步体现了大语言模型在智能家居场景中的应用优势。

与此同时，系统采用模块化设计方式，将语音识别、语义解析以及语音合成等功能进行解耦，便于后续功能扩展与模型升级，为后续智能家居系统智能化能力提升提供了良好的基础。

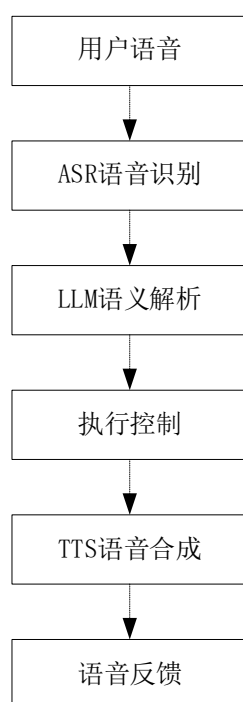


图 5.10 整体语音交互流程图

5.2.4 MQTT 功能实现

在系统实际运行过程中，MQTT 功能主要部署在 ESP32 设备端，用于实现设备与服务器之间的数据交互。

MQTT 通信核心流程代码如算法 5.2 所示。

算法 5.2 MQTT 通信核心流程

```
System_Init();
WiFi_Connect();
while (!MQTT_Connected())
{
    MQTT_Connect(server, port, client_id);
}
MQTT_Subscribe("home/control");
while (1)
{
    if (MQTT_Message_Arrived())
    {
        json_data = MQTT_Receive();
        if (JSON_Parse(json_data))
        {
            device = Get_Device_Name();
            action = Get_Control_Action();
            UART_Send_To_STM32(device, action);
        }
    }
    if (Time_To_Report_Status())
    {
        status_json = Build_Status_Data();
        MQTT_Publish("home/status", status_json);
    }
}
```

设备上电后首先完成网络初始化，并通过 Wi-Fi 接入外部网络环境，为后续通信建立基础条件。在网络连接正常后，设备通过指定的 MQTT 服务器地址（mqtt.xiaozhi.me）发起连接请求，并完成证书校验与身份验证流程。如图 5.11 所示，串口日志显示设备已成功连接至 MQTT 服务器，并完成系统激活，进入正常运行状态。

连接建立后，设备端会订阅对应的控制主题，用于接收来自云端的控制指令。当云端通过 MQTT 发送消息时，ESP32 能够实时接收数据，并对消息内容进行解析处理。系统统一采用 JSON 格式作为通信数据结构，数据中包含设备名称、控制动作以及相关参数信息。

在数据处理流程中，设备端首先对接收到的消息进行格式判断，确认 JSON 结构是否完整，再根据字段内容给 stm32 发出对应的硬件控制指令。

除了控制指令接收之外，设备端还会定时向服务器上报当前运行状态，用于实现设备在线监测与状态同步功能。这些状态信息包括设备运行情况、网络连接状态等内容，有助于云端进行统一管理。

通过 MQTT 通信机制，系统实现了云端与设备端之间的双向数据传输，使语义解析结果能够稳定下发至终端设备，同时也保证了设备状态信息能够及时反馈到云端，从而形成完整的通信闭环。

```
W (8989) Display: SetStatus: 登录服务器...
I (8989) MQTT: Connecting to endpoint mqtt.xiaozhi.me
I (9419) esp-x509-crt-bundle: Certificate validated
I (10209) MQTT: Connected to endpoint
I (10209) Application: Activation done
I (10209) SystemInfo: free sram: 155331 minimal sram: 148155
I (10209) StateMachine: State: activating -> idle
W (10219) Display: ShowNotification: 版本 2.2.3
W (10219) Display: Role:system
W (10219) Display:
I (10229) WifiStation: Setting WiFi power save level: LOW_POWER (MAX_MODEM)
I (10229) wifi:Set ps type: 2, coexist: 0
```

图 5.11 所示 MQTT 连接与系统运行状态串口输出结果

第六章 系统测试与分析

6.1 测试环境与测试方案

6.1.1 测试环境搭建

为了对基于 LLM 的智能家居控制系统在真实使用环境中所表现出的效果做出客观的评价，本文搭建了一个接近实际家庭环境的测试平台。测试环境主要由硬件设备、软件平台以及测试数据集三部分组成，用于模拟智能家居系统的实际运行场景。

硬件部分系统主要用到 ESP32 和 STM32 两个控制芯片。ESP32 利用集成的 WiFi 功能实现云端与本地设备之间的消息通信，STM32 则负责底层设备控制，并通过 GPIO 接口与相关外设模块连接。外接了 LED 灯、温度传感器等家庭环境中常见的设备，实现智能照明设备的开闭控制。其中，LED 灯用于模拟家居照明设备，温度传感器模块用于采集环境温度信息。

软件环境主要采用 Python 3.10、C++ 等开发语言，用于实现服务端逻辑与大语言模型接口调用。语音交互部分使用了小智 IDE 提供的语音识别、语音合成功能，实现用户语音输入识别与系统语音反馈功能。

通信层用 MQTT 协议进行设备间的通信。相较于传统 HTTP 通信方式，MQTT 协议具有轻量化、带宽占用较低以及实时性较好的特点，更符合物联网设备通信场景的需求。系统采用发布订阅的方式来实现控制消息的下发和设备状态的同步，实现云端和终端设备之间数据交互。

系统通过 API 接口调用大语言模型服务。用户输入的自然语言信息先发给服务端，服务端完成语义理解与意图解析后，将结果转换为结构化控制指令。系统之后再根据解析结果向设备端发送控制指令，最终实现从自然语言输入到设备控制执行的完整流程。

表 6.1 系统测试环境配置

类别	配置
主控芯片	ESP32、STM32
开发语言	Python 3.10、C++
通信协议	MQTT
语音模块	小智 IDE
测试设备	小灯、温度传感器等

为了保证测试结果的覆盖性与代表性，本文构建了包含 50 条指令的测试数据集和 20

条非控制类自然语言的测试数据，并对数据集进行了三轮测试，最终取平均结果作为实验数据。指令数据集涵盖了智能家居中的典型交互场景，包括“太暗了”等模糊表达指令，以及“再调低一点”等多轮上下文指令，用于测试系统的语义理解与上下文关联能力。这些测试数据基本覆盖了智能家居日常交互中的常见场景。

表 6.2 列出了部分典型测试样例，包括明确控制指令、模糊表达指令、多轮上下文指令、异常输入指令、非控制类自然语言等类型，能够较好地反映系统在不同交互场景下的实际表现。

表 6.2 测试数据集示例

指令类型	输入示例	预期系统行为
明确指令	打开灯	打开灯
明确指令	空调温度提高 2℃	空调温度上调 2℃
模糊表达指令	太暗了	自动打开灯
模糊表达指令	有点冷	自动提高空调温度
多轮对话指令	打开空调 → 调高一点	在当前温度基础上升高
多轮对话指令	打开灯 → 把它关掉	关闭之前打开的灯
异常指令	空调调至 100 度	拒绝执行异常操作
异常指令	空调调至零下 10 度	拒绝执行异常操作
非控制类自然语言	播放新闻	语音播报热点新闻
非控制类自然语言	你是谁	回答系统预设介绍

6.1.2 测试方案设计

本文主要从功能测试与性能测试两个方面对系统进行验证。

功能测试主要采用黑盒测试方法，从用户实际使用场景出发，对系统功能进行验证。测试过程中，根据不同类型的自然语言指令构建测试数据集，并对语义解析结果进行统计分析，以评估系统在不同场景下的指令识别能力。

当用户输入“有点冷”等带有模糊语义的指令时，系统会结合当前场景信息分析用户意图，并识别其对应的温度调节需求，随后将控制行为与相关设备进行匹配，生成对应的控制指令。多轮对话场景下，系统还要有上下文关联的能力。例如，用户先输入“打开空调”，再输入“调高温度”，系统需要结合前面的对话内容补全当前指令语义，继续对空调设备进行温度调节。

设备控制功能的验证用实际操作来完成，即发送控制指令，观察设备的响应结果。打开灯应该使对应的灯光模块亮起，空调温度升高 2℃应该使温控设备相应调节。除此之外，对多轮对话中出现的指代问题（把它关掉）等，主要对系统识别指代对象、保持上下文准

确性进行测试。

除此之外，系统还对非控制类自然语言交互功能进行了测试。例如用户输入“你是谁”“今天天气怎么样”等开放式语句时，系统能够生成对应文本回复，并通过 TTS 模块完成语音反馈，用于验证系统在开放式自然语言交互场景下的响应能力与交互连贯性。

系统整体测试流程如下图 6.1 所示。系统先接受用户的测试指令，然后用 LLM 对语义进行解析，最后产生对应的结构化控制命令。控制指令经由 MQTT 协议传送到 STM32 控制端，设备执行相应的操作，得到设备的状态信息，从而达到测试结果的记录以及统计的目的。

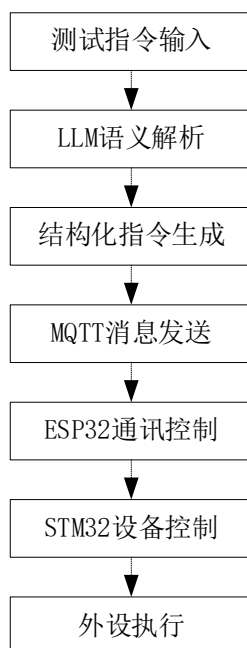


图 6.1 系统整体测试流程图

6.2 功能测试结果与分析

灯光控制测试过程中设备收到控制指令后串口调试信息如图 6.2 示。根据串口输出结果可以看出，设备能接收到 MQTT 控制消息并做出相应的控制动作，说明通信链路是正常的。

系统测试过程中，串口指令示例如图 6.2 所示，图 6.3 展示语音控制系统实测运行结果。其中 A、B、C、D 分别依次对应“太暗了”、“关灯”、“有点热”、“关空调”四条常用语音指令的实际控制测试效果。

```
I (130005) StateMachine: State: listening -> speaking
W (138079) Display: SetStatus: 说话中...
W (138079) Display: Role:user
W (138079) Display:      打开灯。
I (138619) Application: << TURN ON LIGHT.
I (138619) Application: UART Sent: TURN ON LIGHT

W (138619) Display: Role:assistant
W (138619) Display:      TURN ON LIGHT.
I (138819) AudioCodec: Set output enable to true
I (140129) StateMachine: State: speaking -> listening
W (140129) Display: SetStatus: 聆听中...
```

图 6.2 设备串口调试输出

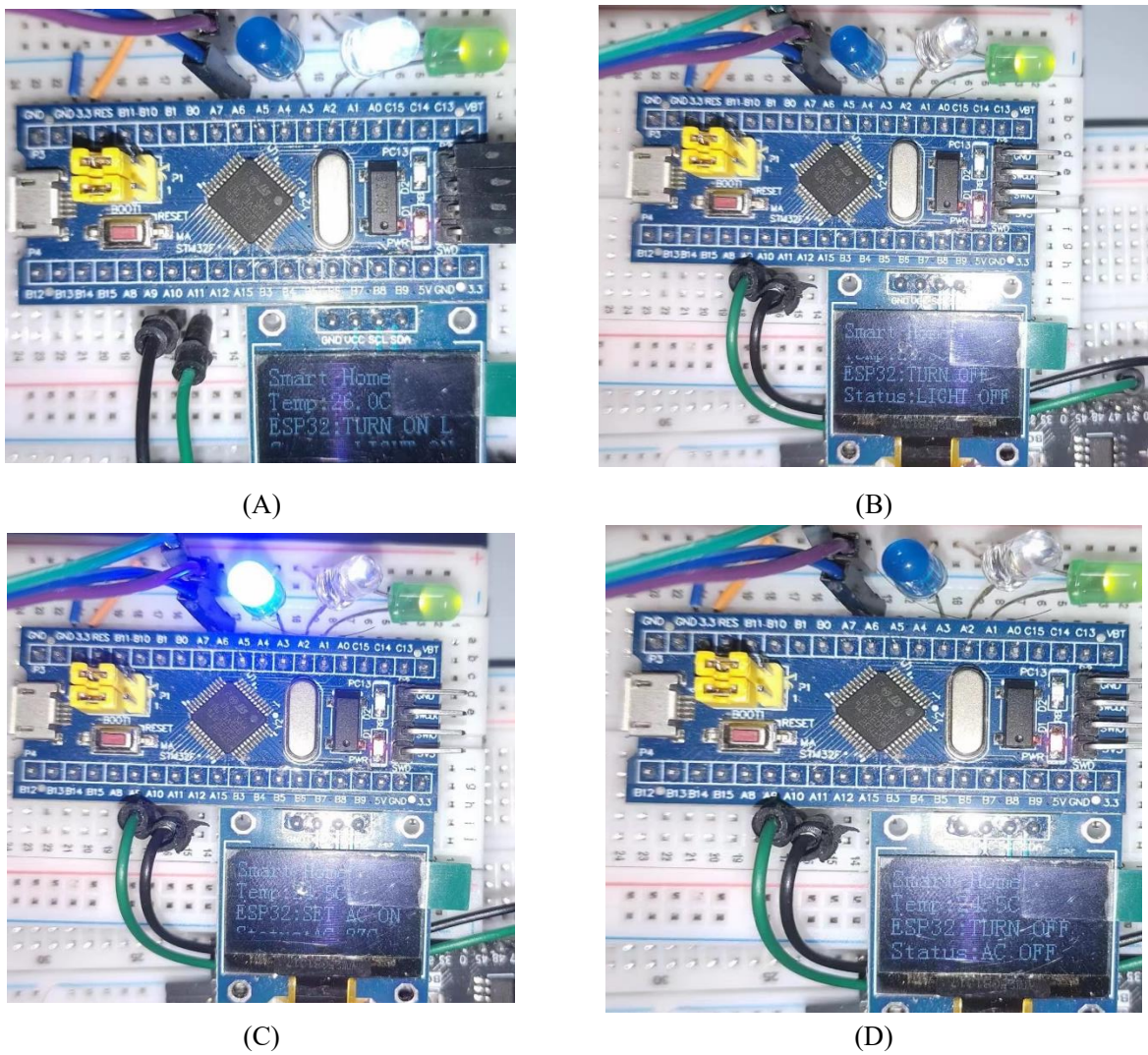


图 6.3 语音控制系统部分测试结果图

同时，为了考察系统对于各种语义场景的指令处理能力，本文利用事先建立好的测试数据集，对语义解析模块做专项测试并加以分析。测试时按照用户输入语句的复杂程度把测试样本分为明确控制指令、模糊语义表达、多轮上下文对话三种类型，分别对系统在不同的场景下解析的效果进行评价测试结果如表 6.3。

表 6.3 不同指令类型测试结果

指令类型	样本数量	解析正确数	准确率
明确指令	20	19	95%
模糊表达指令	15	13	86.7%
多轮对话指令	15	14	93.3%
总体	50	46	92%

测试中，使用关键词匹配的方法与本系统进行了对比实验，对比结果如表 6.4 和图 6.4 所示。

表 6.4 不同方法准确率测试结果

方法	明确指令	模糊表达	多轮对话	总体
关键词匹配	95%	60%	53.3%	72%
本文方法	95%	86.7%	93.3%	92%

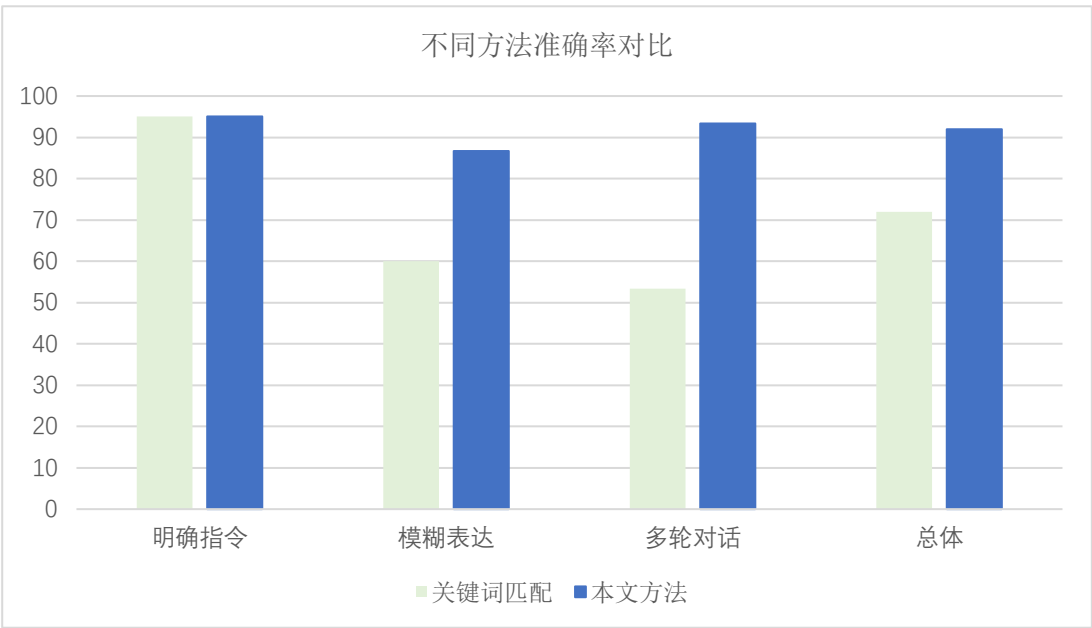


图 6.4 不同方法准确率对比

在非控制类自然语言交互功能测试中，表 6.5 自然语言交互功能的测试结果表明，系统除设备控制功能外，还能够完成基础自然语言交互任务。在非控制类输入场景下，系统能够根据用户输入生成对应回复，并通过语音播报返回结果。实验过程中系统能够保持较好的交互连贯性，说明大语言模型在开放式自然语言交互场景中具有一定可行性。

表 6.5 自然语言交互功能测试结果

输入内容	系统行为
你是谁	返回系统身份介绍
今天天气怎么样	返回天气相关文本回复
播报一条新闻	返回文本信息并语音播报
我有点累	返回安慰文本反馈

从测试结果可以看出，在明确指令测试中系统准确率为 95%，说明系统对于标准化控

制语句有比较稳定的识别和执行能力。模糊表达测试准确率 86.7%，说明系统有语义推理能力，但是隐式语义理解还须加强。多轮对话测试中系统准确率为 93.3%，说明上下文管理机制对语义补全、指代解析起到了较好的作用。

本文方法在三种指令上都比关键词匹配方法有更高的解析准确率，在模糊表达、多轮对话等复杂语义场景下更加明显，说明基于语义理解的方法在复杂语义场景下处理效果更好。

系统大部分测试场景下可以比较稳定地完成语义解析和设备控制，在一些比较复杂的模糊语义场景中还会出现误判的情况。用户输入“有点闷”时，系统不能将用户想打开灯和降低空调温度区分出来，在部分多轮对话场景中，当对话轮次较多的时候，系统就会出现上下文关联错误的情况。

造成上述问题的主要原因就是目前系统上下文记忆长度小、测试阶段使用的 LLM 接口没有对智能家居场景进行专门的微调，造成部分隐式意图理解能力不足。后续工作可以加上长期上下文记忆，用领域数据来优化。

除此之外，在对异常指令进行测试的时候，当空调温度被设置为超出安全范围的异常值时，系统会自动拒绝执行相关指令，并给出安全提示信息，以防止异常控制行为对实际设备造成影响。

6.3 性能测试结果与分析

性能评价主要是对系统的响应时间进行测试。响应时间指的是设备接到指令之后，直到执行完毕并给出反馈为止所耗费的时间。WiFi 网络正常工作的情况下做了多次测试，统计结果如表 6.6 所示。

表 6.6 性能测试结果	
测试指标	数值
最小响应时间	1.3 s
最大响应时间	3.1 s
平均响应时间	1.8s

从测试结果可知系统平均响应时间为 1.8s，基本可以满足智能家居交互的要求。

除此之外，在测试过程中系统未出现明显的异常中断或者指令执行失败情况，说明系统有较好的运行稳定性。同时由于云端语义解析与边缘设备执行相结合，系统既保持了语义解析的能力又很好地控制了整体的延迟。

6.4 本章小结

本章主要对基于 LLM 的智能家居控制系统进行测试以及性能分析。建立包含明确指令、模糊表述、多轮对话等场景的数据集，在实际的硬件环境下对功能、性能做了测试。

根据综合测试结果可知，本文系统在设置好的控制场景下解析准确率高，模糊表达、多轮对话等情况下也有较好的语义理解能力，准确率为92%。相较于传统关键词匹配方法，本文方法在复杂语义交互场景下具有更好的适应能力。

根据性能测试结果可知系统平均响应时间大约为 1.8s，可以满足智能家居交互的需求。就整体而言，系统在大部分的测试中是稳定的，语义解析的准确率以及响应速度也基本达到了预期。

第七章 总结与展望

7.1 研究工作总结

针对传统智能家居系统在自然语言交互过程中语义理解能力不足、模糊指令处理能力较弱、多轮对话连续性较差等问题,本文对大语言模型智能家居控制系统进行设计和实现。围绕自然语言交互需求,系统结合语义理解、上下文关联以及安全校验等功能,实现了对智能家居设备的自然语言控制,相比传统规则匹配方式具有更好的交互适应能力。

本文采用五层三端的分层系统结构,即用户交互层、LLM 推理层、数据存储层、设备管理与驱动层、硬件执行层。系统采用云边协同的方式,把计算密集型任务放在云端,硬件控制在边缘侧完成,减小了终端设备的运算压力。分层设计既能够发挥大语言模型在语义理解方面的优势,也能够满足智能家居场景下的交互需求。

在语义解析方面,针对智能家居场景下口语化的输入特点,采用基于语义约束的指令解析方法。通过事先设定好的 Prompt 模板对模型生成的格式加以约束,使返回结果能够保持统一的结构形式。系统结合关键词匹配方法对设备名称、控制动作等信息进行提取,从而提高了指令解析结果的稳定性。同时,系统通过上下文管理机制关联历史对话信息,提高了连续对话场景下的语义理解能力。

从安全机制上讲,由于大语言模型存在着异常输出的问题,因此设计了双层安全控制机制。系统先通过关键词规则过滤高危操作,再结合语义分析方法对潜在异常指令进行进一步识别与拦截。通过上述安全控制机制,系统在保证响应速度的基础上,进一步提升了设备控制过程中的安全性与稳定性。

本文以 ESP32、STM32 为边缘控制节点,用 MQTT 协议实现设备间的通信,完成设备通信功能的开发。系统采用统一的结构化控制指令,使云端语义解析结果可以和边缘设备进行数据交换。系统集成了 ASR 和 TTS 模块,实现语音输入、语义解析、设备执行、语音反馈的完整交互过程。同时,系统还支持基础自然语言对话功能,在非控制类输入场景下能够生成对应文本回复,并通过语音方式进行反馈,提升了整体交互体验。

本文创建了包含明确指令、模糊表达、多轮对话、异常输入等指令的测试集,对系统的语义解析准确率、响应速度、安全校验能力做了测试。测试结果表明,系统整体指令解析准确率为 92%,端到端平均响应时间约为 1.8s,在模糊语义输入和连续对话场景下,系统能够结合上下文信息完成语义补全与指令解析,相较于传统规则匹配方式具有更好的交互适应能力。系统对于异常的控制指令也会做出识别和拦截,防止高风险操作直接进入设

备执行阶段。实验结果表明，系统能够较好地完成智能家居场景下的自然语言控制任务，并具备一定的实际应用价值。

综上所述，本文主要研究工作及成果可以概括为五个方面，即

(1) 提出了一个面向智能家居场景的语义约束指令解析方法。采用 Prompt 约束加关键词校验的方法，可以实现自然语言到结构化控制指令稳定转化。

(2) 设计云边协同的智能家居分层控制架构。将云端语义推理和边缘设备控制分层协同起来，将计算任务与设备控制任务分别部署，提升了系统实时性与工程适配能力。

(3) 构建了双层安全控制机制。按照关键词规则过滤、语义安全判定策略，对异常指令、潜在风险操作进行了识别和拦截，使系统运行过程更加安全、稳定。

(4) 完成系统的原型开发和实验验证。基于 ESP32 和 STM32 平台搭建了完整的智能家居原型系统，用多场景测试验证了系统在语义理解、设备控制、安全防护等各方面的实际应用效果。

(5) 实现基础自然语言交互功能。除设备控制外，系统还支持非控制类自然语言输入的基础对话回复功能，能够结合大语言模型生成对应文本反馈，并通过语音播报实现自然交互，提升了系统自然交互能力与用户使用体验。

7.2 研究不足与未来展望

虽然本文完成了基于大语言模型的智能家居控制系统的设计，但是在实际的应用以及系统的工程化上还有待于进一步的改进。

目前系统主要依靠云端大语言模型来完成语义解析任务。该方法虽然可以得到较好的自然语言理解能力，但是对网络环境要求较高。当网络状况不好或者处于离线状态的时候，系统响应速度、稳定性都会受到影响。后续可以结合轻量化模型与边缘部署方案，在本地设备端进行部分语义推理，从而减小对云端算力、网络连接的依赖程度。

其次，目前系统功能主要是单设备控制、基本的多轮对话场景，对于多设备联动、复杂场景控制的支持还比较少。智能家居实际应用时，用户要借助“回家模式”或者“睡眠模式”这些指令来操控灯光，空调以及窗帘这些设备。之后可以进一步加强场景化联动控制能力，提高系统的整体实用性。

系统虽然已经加入关键词过滤、语义校验等安全控制手段，但是对一些复杂的情况仍然存在欠缺。隐式表达、绕过式控制语句时，目前的规则机制很难完全准确地识别出来。可以使用更优的语义风险分类、行为检测等方法来提高系统在复杂交互环境下的安全控制

能力。

除此之外，目前系统只根据用户当前输入控制设备，对于用户长期使用习惯的学习与分析能力仍然较弱。之后可以利用用户行为建模以及个性化推荐机制，使得系统根据用户的过去操作习惯给出控制建议，进而提升系统的智能化程度和用户交互体验。

综上所述，大语言模型和物联网技术不断发展，智能家居系统将朝着更智能、个性化、自主化方向发展，本文的研究结果对大语言模型在智能家居控制场景中的应用具有一定参考价值。

参考文献

- [1] Liu Y, Liu W G. Research on the application status and future trend of Internet of Things technology in smart city[EB/OL]. <https://journals.viserdata.com/index.php/sca/article/view/15082>, 2026-04-10.
- [2] 刘峒, 李妍, 叶森. 国内外智能家居研究进展与趋势——基于 2013—2023 年核心期刊文献的 CiteSpace 可视化分析[J]. 木材科学与技术, 2024, 38(3): 78-87.
- [3] Zhao W X, Zhou K, Li J Y, et al. A survey of large language models[EB/OL]. <https://arxiv.org/abs/2303.18223>, 2026-04-10.
- [4] Rivkin D, Hogan F, Feriani A, et al. AIoT smart home via autonomous LLM agents[J]. IEEE Internet of Things Journal, 2024, 12(3): 2458-2472.
- [5] 车万翔, 窦志成, 冯岩松, 等. 大模型时代的自然语言处理: 挑战, 机遇与发展[J]. 中国科学: 信息科学, 2023, 53(9): 1645-1687.
- [6] 奚雪峰, 周国栋. 面向自然语言处理的深度学习研究[J]. 自动化学报, 2016, 42(10): 1445-1465.
- [7] Sharif K, Tenbergen B. Smart home voice assistants: a literature survey of user privacy and security vulnerabilities[J]. Complex Systems Informatics and Modeling Quarterly, 2020(24): 15-30.
- [8] Devi M. Context-Aware Dialogue Systems for Human-Computer Interaction Applications[J]. International Journal of Science, Research and Technology, 2022, 5(3): 7769-7776.
- [9] 雷雨琦. 基于深度学习的智能家居控制系统聊天机器人研究[D]. 武汉: 华中科技大学, 2023.
- [10] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[J]. Advances in Neural Information Processing Systems, 2017, 30: 5998-6008.
- [11] Devlin J, Chang M W, Lee K, et al. BERT: Pre-training of deep bidirectional transformers for language understanding[C]//Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers). 2019: 4171-4186.
- [12] Ouyang L, Wu J, Jiang X, et al. Training language models to follow instructions with human feedback[J]. Advances in Neural Information Processing Systems, 2022, 35: 27730-27744.
- [13] Radford A, Narasimhan K, Salimans T, et al. Improving language understanding by generative pre-training[R]. San Francisco: OpenAI, 2018.
- [14] Brown T, Mann B, Ryder N, et al. Language models are few-shot learners[J]. Advances in Neural Information Processing Systems, 2020, 33: 1877-1901.
- [15] Wei J, Tay Y, Bommasani R, et al. Emergent abilities of large language models[EB/OL]. <https://arxiv.org/abs/2206.07682>, 2026-04-10.
- [16] Li S, Guo Y, Yao J, et al. HomeBench: Evaluating LLMs in smart homes with valid and invalid instructions across single and multiple devices[C]//Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). 2025: 12230-12250.
- [17] Fielding R T. Architectural styles and the design of network-based software architectures[D]. Irvine: University of California, 2000.
- [18] O'Reilly T. What is Web 2.0[EB/OL]. <https://www.oreilly.com/pub/a/web2/archive/what-is-web-20.html>, 2026-04-10.
- [19] Berners-Lee T, Cailliau R, Luotonen A, et al. The world-wide web[J]. Communications of the ACM, 1994, 37(8): 76-82.
- [20] OASIS Standard. MQTT Version 5.0[S]. Burlington: OASIS, 2019.
- [21] Makatita F D, Hakim N F A. MQTT protocol-based ESP-32 smarthome with multi-sensor recognition[J]. Journal of Electrical, Electronic, Information and Communication Technology, 2024, 6(1): 29-36.
- [22] 李伟, 贾奇伟, 劳春峰, 等. 基于 LLM 的家电智能交互控制系统研究[J]. 家电科技, 2024(S1): 125-

129.

- [23] 王海坤, 潘嘉, 刘聪. 语音识别技术的研究进展与展望[J]. 电信科学, 2024, 34(2): 1-11.
- [24] Shen J, Pang R, Weiss R J, et al. Natural TTS synthesis by conditioning WaveNet on mel spectrogram predictions[C]//2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). Calgary: IEEE, 2018: 4779-4783.
- [25] Shen K, Ju Z, Tan X, et al. NaturalSpeech 2: Latent diffusion models are natural and zero-shot speech and singing synthesizers[EB/OL]. <https://arxiv.org/abs/2304.09116>, 2026-04-10.
- [26] Hakkani-Tür D, Tür G, Heck L, et al. Multi-domain joint semantic frame parsing using bidirectional RNN-LSTM[C]//Proceedings of the Annual Conference of the International Speech Communication Association. 2016: 715-719.
- [27] Goo C W, Gao G, Hsu Y K, et al. Slot-gated modeling for joint slot filling and intent prediction[C]//Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies. 2018: 753-757.
- [28] Sun C, Huang S, Pompili D. LLM-based multi-agent decision-making: Challenges and future directions[J]. IEEE Robotics and Automation Letters, 2025, 10(6): 5681-5688.
- [29] Bordes A, Boureau Y L, Weston J. Learning end-to-end goal-oriented dialog[C]//International Conference on Learning Representations. 2017.
- [30] Young T, Cambria E, Chaturvedi I, et al. Recent trends in deep learning based natural language processing[J]. IEEE Computational Intelligence Magazine, 2018, 13(3): 55-75.
- [31] Mizrahi M, Kaplan G, Malkin D, et al. State of what art? a call for multi-prompt llm evaluation[J]. Transactions of the Association for Computational Linguistics, 2024, 12: 933-949.

致谢

敲下论文最后一个句点之后，回望自己从一行行代码、一次次调试到整篇论文成型的日夜，心情十分激动。在此我谨对本课题研究中给予我帮助和支持的人表示最真挚的感谢。

首先要感谢我的导师。课题选题、系统方案设计、论文修改完善过程中，导师一直对我的工作予以悉心教导。无论是从研究思路、技术实现还是论文结构安排上，导师都给了我许多宝贵的建议。导师严谨认真的治学态度，对毕业设计的全过程起到了非常大的作用。

感谢和我一起工作过的同学。忘不了我们一起解决多设备硬件联调 bug 的时光，我们一起加班加点。你们的一点一滴的独到见解、一次次的援助都是我陷入困境时寻找新思路的钥匙，我们一起走过的日子我将永远铭记。

同时，也要感谢所有开源社区的工作者，以及智能家居、大语言模型领域的所有科研前辈。是你们的研究成果与开源项目，为我的课题搭建了坚实的阶梯，让我得以站在前人的肩膀上，完成这一次小小的探索。

论文有终章，求知无止境。未来，我将带着这份收获与感动，继续在技术领域深耕，以微薄之力，为智能家居的智能化升级、为大语言模型的场景化落地，贡献自己的一份力量。